

4 工厂模式

用OO的精髓做烘焙



准备烘焙一些松耦合OO设计。制造对象，不仅只有使用new操作符。你将会学习到，实例化这个活动不应该总是公开进行，因为经常会导致耦合问题。我们不喜欢这样，对吧？阅读下去，你会了解工厂模式如何能把你从令人难堪的依赖中拯救出来。

OK, 已经过了三章, 你还没回答关于new的问题。我们不应该针对实现编程, 但每次使用new时, 不正是在针对实现编程吗?



当你看到“new”，会想到“具体”。

是的, 当你使用new 操作符时, 你无疑是在实例化一个具体类, 因此, 这肯定是一个实现而不是接口。你观察得很仔细: 把你的代码绑在具体类上, 会使得代码更脆弱, 更缺乏弹性。

```
Duck duck = new MallardDuck();
```

我们要用抽象类型来保持代码弹性。

但我们不得不创建一个具体类的实例!

当我们有一整群相关的具体类时, 经常会这样编码:

```
Duck duck;  
  
if (picnic) {  
    duck = new MallardDuck();  
} else if (hunting) {  
    duck = new DecoyDuck();  
} else if (inBathTub) {  
    duck = new RubberDuck();  
}
```

我们有一堆不同的鸭子类, 直到运行时才会知道需要实例化哪一个。

这里有一些要实例化的具体类, 实例化哪一个, 要在运行时由一些条件决定。

当你看到这样的代码, 你就知道, 当有变化或需要扩展时, 你将不得不再次打开这份代码, 检查需要添加(或删除)的地方。通常, 这种代码会分布在系统的各处, 使得维护和更新更加困难, 而且更容易出错。

但是，某些时候你不得不创建对象，Java只给了我们一种创建对象的方式，对吧？那是怎么回事呢？

“new”有什么问题？

在技术上，new操作符没有错。毕竟，它是大多数现代面向对象语言的基础部分。真正的肇事者是我们的老朋友“变化”以及它对使用new的影响。

通过对接口编码，你知道可以隔离以后系统可能发生的许多变化。为什么？如果你的代码针对接口编写，那么通过多态，它可以和任何实现这个接口的新类一起工作。但是，当你的代码利用了大量的具体类，你是在自找麻烦，因为一旦加入新的具体类，就必须改变代码。因此，换句话说，你的代码不是“对修改关闭”。为了用新的具体类型扩展你的代码，你不得不再次打开它。

因此，该怎么办？这种时候你可以求助于OO设计原则，在其中寻找线索。记住，我们的第一个原则用来处理变化，并引导我们识别变化的方面并把它们和不变的方面分离。

记得，设计应该“对扩展开放，对修改关闭”。回顾一下第3章吧。



怎样把应用中所有实例化具体类的代码拿出来分离，或者封装起来，这样不会影响应用的其他部分？

识别变化的方面

假设你有一间比萨店，作为对象村最新潮的比萨店老板，你写了一些像这样的代码：

```
Pizza orderPizza() {
    Pizza pizza = new Pizza();

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```



为了弹性，我们真的需要这个类成为一个抽象类或者接口，但不幸的是，这两个我们不能直接实例化。

但你需要更多类型的比萨……

然后你添加一些代码，决定合适的比萨类型，然后再制作比萨：

```
Pizza orderPizza(String type) {
    Pizza pizza;
```

把比萨的类型传给
orderPizza。

```
    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    } else if (type.equals("greek")) {
        pizza = new GreekPizza();
    } else if (type.equals("pepperoni")) {
        pizza = new PepperoniPizza();
    }
}
```

基于比萨的类型，我们实例化正确的具体类，然后将其赋值给pizza实例变量。注意，这里的每个比萨都必须实现Pizza接口。

```
    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

一旦我们有了一个Pizza，我们做前期准备（你知道的，擀揉面团、加酱汁、加配料），然后烘焙，切片，装盒！

每个Pizza子类型（CheesePizza、GreekPizza等）知道如何准备自己。

但压力来自添加更多的比萨类型

你意识到，所有竞争对手已经在菜单中加入一些流行风味的比萨：蛤蜊（Clam）比萨、素食（Veggie）比萨。显然，你需要跟上竞争态势，因此也要把这些风味加进菜单。而最近希腊（Greek）比萨销售不佳，因此你决定将它从菜单中除名：

```
Pizza orderPizza(String type) {
    Pizza pizza;

    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    } else if (type.equals("greek")) {
        pizza = new GreekPizza();
    } else if (type.equals("pepperoni")) {
        pizza = new PepperoniPizza();
    } else if (type.equals("clam") {
        pizza = new ClamPizza();
    } else if (type.equals("veggie") {
        pizza = new VeggiePizza();
    }

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

这份代码没有对修改封闭。
如果比萨店改变其比萨供应，我们就得打开这份代码来修改。

这是变化的地方。
随着时间推移，
披萨选项会变化，
你不得不一遍遍修改这里的代码。

这里是我们期望保持不变的地方。
因为披萨的准备、烘焙、包装的大部分工作多年来都保持不变。因此，我们不期望这些代码有变化，只改变要操作的比萨。

显然，关于哪个具体类被实例化的代码才是真正搞乱orderPizza()方法的罪魁祸首，它使得代码无法对修改关闭。但现在我们知道哪些会改变，哪些不会改变，可能是时候把它封装起来了。

封装对象的创建

因此现在我们知道最好把对象创建移到orderPizza()方法外面。但怎么做？嗯，我们打算把创建代码移到另一个对象中，该对象只关心创建比萨。

```
Pizza orderPizza(String type) {
```

```
    Pizza pizza;
```

```
    pizza.prepare();
```

```
    pizza.bake();
```

```
    pizza.cut();
```

```
    pizza.box();
```

```
    return pizza;
```

```
}
```

首先，我们把对象创建的代码从orderPizza()方法拉出来。

这里该怎么写？

```
if (type.equals("cheese")) {
    pizza = new CheesePizza();
} else if (type.equals("pepperoni")) {
    pizza = new PepperoniPizza();
} else if (type.equals("clam")) {
    pizza = new ClamPizza();
} else if (type.equals("veggie")) {
    pizza = new VeggiePizza();
}
```

然后，我们把这部分的代码放进一个只操心如何创建比萨的对象。如果任何其他对象需要创建比萨，就找这个对象。



简单工厂

我们给这个新对象起了个名字：工厂。

工厂处理对象创建的细节。一旦我们有了一个SimplePizzaFactory，orderPizza()方法就变成这个对象的客户。任何时候它需要比萨，就叫比萨工厂做一个。那些orderPizza()方法需要知道希腊比萨或者蛤蜊比萨的日子就去不复返了。现在orderPizza()方法只关心它得到了一个实现Pizza接口的比萨，这样它就可以调用prepare()、bake()、cut()和box()。

还有一些细节需要补充，例如，原来orderPizza()方法中创建的代码，现在怎么写？我们来为比萨店实现一个简单的工厂，来找出答案……

建立一个简单的比萨工厂

我们从工厂本身开始。我们打算定义一个类，这个类封装所有比萨的对象创建。如下所示：

这是我们的新类SimplePizzaFactory。
它一生只做一件事：为它的客户创建比萨。

```
public class SimplePizzaFactory {
```

```
    public Pizza createPizza(String type) {
        Pizza pizza = null;
```

```
        if (type.equals("cheese")) {
            pizza = new CheesePizza();
        } else if (type.equals("pepperoni")) {
            pizza = new PepperoniPizza();
        } else if (type.equals("clam")) {
            pizza = new ClamPizza();
        } else if (type.equals("veggie")) {
            pizza = new VeggiePizza();
        }
        return pizza;
    }
}
```

首先，我们在这个工厂内
定义一个createPizza()方法。
所有客户用这个方法来实现
例化新对象。

这是我们从orderPizza()
方法中移过来的代码。

这个代码依然是以比萨的类型为参数，
和原始版本的orderPizza()方法一样。

没有蠢问题

问：这么做有什么好处？看起来我们只是把问题推到另一个对象而已。

答：要记住一件事，SimplePizzaFactory可以有多个客户。虽然我们只看到了orderPizza()方法，但是，可能还会有PizzaShopMenu类用该工厂来取得比萨的当前描述和价格。还可能有HomeDelivery类，用另外的方式处理比萨，但它也是工厂的客户。

因此，通过把比萨的创建封装进一个类，当实现变化时，我们只要改这个地方。

还有，别忘了，我们也正要把具体实例化的部分从客户代码移除。

问：我见过一个类似的设计，工厂被定义为一个静态方法。有什么区别？

答：定义简单工厂为静态方法是一个常用技巧，常称为静态工厂。为什么要用静态方法？因为你不需要使用创建方法来实例化一个对象。但这也有缺点，你不能子类化并改变创建方法的行为。

重做PizzaStore类

现在是时候修正我们的客户代码了。我们要做的是，依靠工厂创建比萨。修改如下：

```
public class PizzaStore {
    SimplePizzaFactory factory;

    public PizzaStore(SimplePizzaFactory factory) {
        this.factory = factory;
    }

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = factory.createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
    }

    // 其他方法放这里
}
```

首先，我们给PizzaStore一个SimplePizzaFactory的引用。

PizzaStore在构造器中获得传给它的工厂。

orderPizza()方法通过简单地传递订单类型，使用工厂来创建比萨。

注意，我们把new操作符替换成工厂对象的createPizza方法。这里不再有具体实例化！



我们知道对象组合允许我们在运行时动态地改变行为，因为可以更换不同的实现。在PizzaStore中怎样才能应用这一点？什么工厂的实现能够被更换？

。不知道你怎么想，我想到的是纽约、芝加哥、加州风味的比萨工厂（别忘了还有纽黑文）。

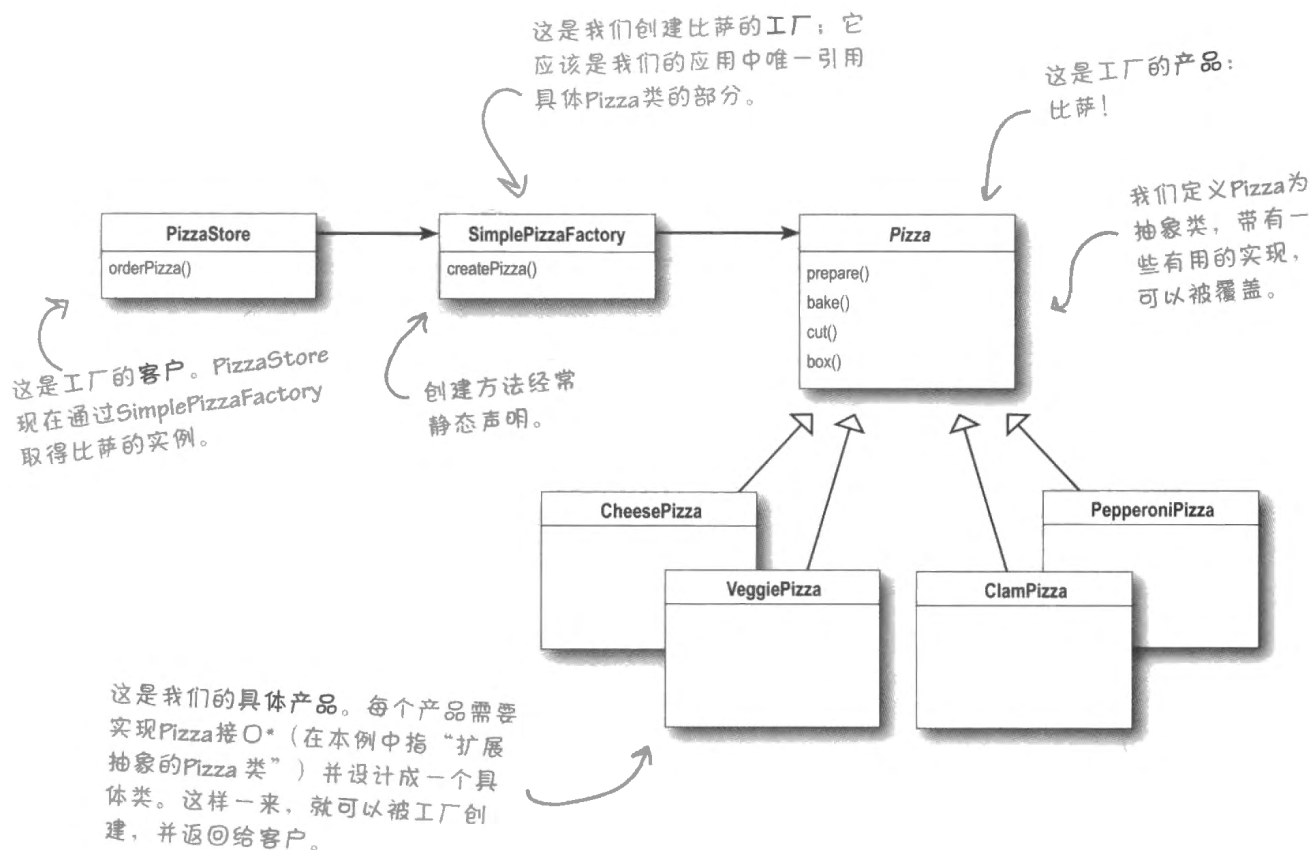
定义简单工厂

简单工厂其实不是一个设计模式，更多是一种编程习惯。但它经常被使用，因此我们给它一个Head First 模式荣誉奖。一些开发人员的确把这个习惯误认为是工厂模式，不过，下一次碰到这种场合，就可以微妙地展示一下你懂得其中区别，但不要过分炫耀。



模式
荣誉奖

简单工厂不是一个真正的模式，但这并不意味着我们不应该好好了解其中道理。我们看看新比萨店的类图：



把简单工厂看作是热身。接下来，我们将探索两个重量级的模式。它们都是工厂。但是别担心，还有更多的比萨要来！

*另一个提醒：在设计模式中，用语“实现一个接口”并不总是意味着“写一个类，在类声明中通过使用implements关键词来实现某个Java接口”。从这个用语的一般用法来说，一个具体类实现了一个来自超类型（可以是抽象类或接口）的方法，依然可以看作“实现这个超类型的接口”。

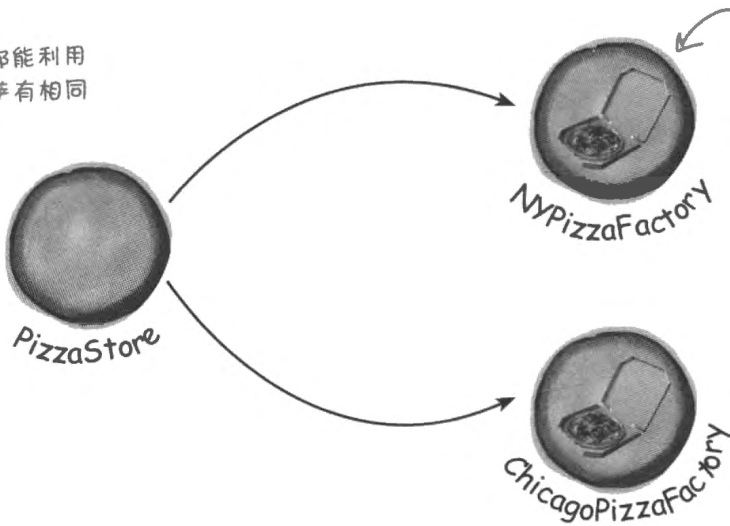
连锁加盟比萨店

对象村比萨店生意很好，打败了竞争对手，现在每个人都希望自家附近有一家比萨店。作为连锁加盟的经销商，你要确保连锁加盟运营的质量，因此希望加盟店都使用你那些经过时间考验的代码。

但是区域的差异呢？根据加盟店地点以及该地区比萨美食家的口味，每家加盟店可能要提供不同风味的比萨（比如纽约、芝加哥、加州）。

是的，美国的不同区域需要提供的比萨风味差别很大，从芝加哥的深盘比萨到纽约的薄皮比萨，再到加州的薄脆比萨（据说有一些会覆盖水果或坚果）。

你希望所有加盟店都能利用你的代码，好让比萨有相同的准备方式。



一家加盟店需要做纽约风味比萨的工厂：薄皮、美味酱汁，只加一点芝士。

另一家加盟店需要做芝加哥风味比萨的工厂：他们的顾客喜欢厚皮、重口味酱汁和大量芝士。

我们已经看到了一种做法……

如果我们把SimplePizzaFactory拿出来，创建三个不同的工厂：NYPizzaFactory、ChicagoPizzaFactory和CaliforniaPizzaFactory，然后，我们只要把PizzaStore 和适合工厂组合起来，加盟店的问题就搞定了。

我们来看看是什么样子……

```
NYPizzaFactory nyFactory = new NYPizzaFactory();
```

```
PizzaStore nyStore = new PizzaStore(nyFactory);
```

```
nyStore.orderPizza("Veggie");
```

这里我们创建做纽约风味比萨的工厂。

然后我们创建一个PizzaStore，并给它传递一个纽约工厂的引用。

……当我们制作比萨时，得到了纽约风味的比萨。

```
ChicagoPizzaFactory chicagoFactory = new ChicagoPizzaFactory();
```

```
PizzaStore chicagoStore = new PizzaStore(chicagoFactory);
```

```
chicagoStore.orderPizza("Veggie");
```

芝加哥比萨店也类似。我们为芝加哥比萨创建一个工厂，并创建一个比萨店和芝加哥工厂组合。制作比萨时，就得到了芝加哥风味比萨。

但你要多一些质量控制……

因此你尝试一下SimpleFactory的点子，看看市场的反应，发现加盟店在使用你的工厂来创建比萨，但流程中的其他部分，开始采用他们自己自创的过程：烘焙的做法有些差异、忘了切片、使用第三方的盒子。

重新思考一下这个问题，你了解到，你真正要做的是：创建框架，把加盟店和比萨创建绑在一起，但仍然允许保持有弹性。

在我们之前的代码中，在SimplePizzaFactory之前，我们把制作比萨的代码绑到PizzaStore，但这样做没有弹性。因此，怎样才能吃掉比萨又保有比萨呢？

我做比萨有些年头了，因此想在**PizzaStore**的流程中，加入自己的“改良”……

这不是你想要的好加盟店。他在比萨里放了什么，你不会想知道的。



为比萨店而做的框架

有个方法可让比萨制作活动局限于PizzaStore类，同时给予加盟店自由来拥有他们自己的地区风味。

我们打算做的是，把createPizza()方法放回PizzaStore，但这一次作为一个抽象方法，然后为每个区域风味创建一个PizzaStore子类。

首先，我们来看看PizzaStore的变化：

PizzaStore现在是抽象的（原因见下）。

```
public abstract class PizzaStore {  
  
    public Pizza orderPizza(String type) {  
        Pizza pizza;  
  
        pizza = createPizza(type);  
  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
  
        return pizza;  
    }  
  
    abstract Pizza createPizza(String type);  
}
```

现在createPizza从工厂对象移回PizzaStore。

所有这些都相同……

现在，把工厂对象移到这个方法。

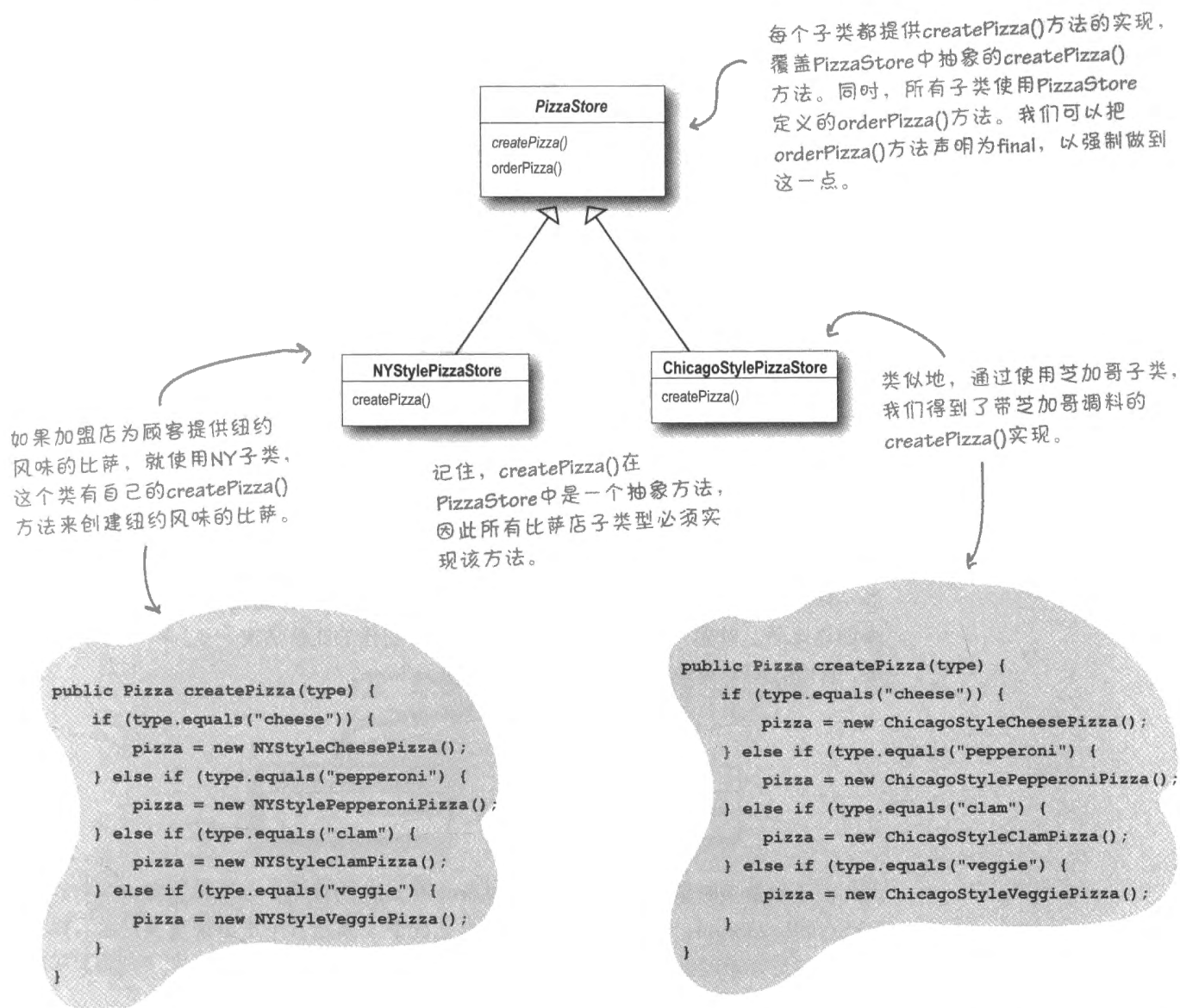
现在，在PizzaStore中，我们的“工厂方法”是抽象的。

现在我们有了一个等待着子类的店。我们打算让每个区域类型有一个子类（NYPizzaStore、ChicagoPizzaStore、CaliforniaPizzaStore），每个子类将决定比萨由什么组成。我们看看怎么运作。

允许子类决定

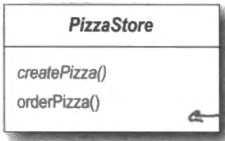
记住，在orderPizza()方法中，比萨店已经有了一个很不错的订单系统。你要确保所有加盟店在这上面保持一致。

各个区域比萨店之间的差异，在于他们制作比萨的风味（纽约比萨的饼薄、芝加哥比萨的饼厚等），我们打算把所有这些变化放进createPizza()方法，让其负责创建正确种类的比萨。做法是让PizzaStore的各个子类来定义自己的createPizza()方法。因此，会有一些PizzaStore的具体子类，每个有自己的比萨变体，但都适合PizzaStore框架，依然使用调试好的orderPizza()方法。



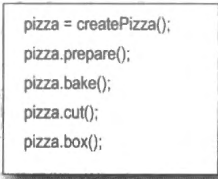
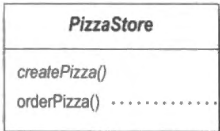


好吧，从PizzaStore的orderPizza()方法的视角考虑一下：它在抽象的PizzaStore中定义，但具体类型只在子类中创建。



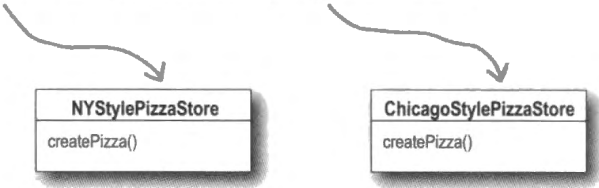
orderPizza()方法在抽象的PizzaStore中定义，而不是在子类中。因此，该方法不知道哪个子类实际运行代码并制作比萨。

现在，再往前迈一小步，orderPizza()方法对Pizza对象做了许多事情（例如：准备、烘焙、切片、装盒），但由于Pizza是抽象的，orderPizza()不知道涉及了哪一个实际的具体类。换句话说，这就是被解耦了！



orderPizza()调用createPizza()来确实获得一个比萨对象。但获得哪一种比萨，orderPizza()方法不能决定；它不知道如何做。那么，谁决定呢？

当orderPizza()调用createPizza()时，某个子类将被调用，行动起来创建比萨。做哪一种比萨？嗯，由你下单时选择的比萨店决定，NYStylePizzaStore或者ChicagoStylePizzaStore。



因此，子类是实时做决定吗？不，但从orderPizza()的视角，如果你选择NYStylePizzaStore，这个子类就决定做什么比萨。因此，子类不是真正“决定”（是你通过选择比萨店决定），但它们确实决定了做哪种比萨。

我们来做一家比萨店

开加盟店有它的好处。你可以免费获得PizzaStore的所有功能。所有区域店只需要继承PizzaStore，并提供实现自己的比萨风味的createPizza()方法。我们关注一下三个比较重要的加盟店的比萨风味。

这里是纽约区域风味：

createPizza()返回一个Pizza，子类全权负责该实例化哪一个具体Pizza。

NYPizzaStore扩展PizzaStore，因此它继承orderPizza()方法（以及其他的方法）。

```
public class NYPizzaStore extends PizzaStore {
```

```
    Pizza createPizza(String item) {
        if (item.equals("cheese")) {
            return new NYStyleCheesePizza();
        } else if (item.equals("veggie")) {
            return new NYStyleVeggiePizza();
        } else if (item.equals("clam")) {
            return new NYStyleClamPizza();
        } else if (item.equals("pepperoni")) {
            return new NYStylePepperoniPizza();
        } else return null;
    }
}
```

我们必须实现createPizza()，因为它在PizzaStore中是抽象的。

这是创建具体类的地方。我们对每一种Pizza类型创建纽约风味。

* 注意，超类的orderPizza()方法不知道哪种Pizza正在创建；它只知道它可以对这个比萨准备、烘焙、切片、装盒！

一旦PizzaStore子类建立，是时候考虑订购一两个比萨了。但这么做之前，为什么你不在下一页尝试建造一下芝加哥风味以及加州风味的比萨店？



削尖你的铅笔

我们已经搞定了NYPizzaStore，再搞定两个就可以准备开加盟店了。在下面编写芝加哥和加州的比萨店：

声明一个工厂方法

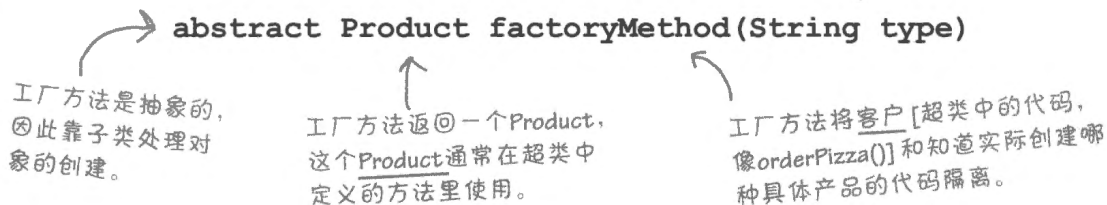
对PizzaStore类做一些转换，从由一个对象处理具体类的实例化，到由一个子类集合来承担责任。我们来看得更仔细一点：



Code up Close

工厂方法处理对象的创建，并将对象创建封装在子类中，使得超类中的客户代码从子类的对象创建代码解耦。

工厂方法可能参数化（也可能不）来在产品的若干变体中做选择。



我们来看其中机制：用披萨工厂方法订购披萨



那么，他们怎么下单的？

- 1 首先，Joel和Ethan需要一个PizzaStore的实例。Joel需要实例化一个ChicagoPizzaStore，Ethan需要一个NYPizzaStore。
- 2 手上了有了PizzaStore，Joel和Ethan分别调用orderPizza()方法，并传递他们想要的披萨类型（芝士、素食等）。
- 3 为了创建披萨，createPizza()方法被调用，该方法在两个子类 NYPizzaStore和ChicagoPizzaStore中定义。在定义时，NYPizzaStore实例化一个纽约风味的披萨，ChicagoPizzaStore实例化一个芝加哥风味的披萨。两种情况下，该Pizza都被返回给orderPizza()方法。
- 4 orderPizza()方法不知道哪一种披萨被创建了，但知道它是一个披萨，并且将其准备、烘焙、切片、装盒给Ethan和Joel。

我们来追查一下，这些比萨是怎样
真实根据订单制作的……

幕后花絮



1 我们来跟踪Ethan的订单：首先，我们需要一个NYPizzaStore：

```
PizzaStore nyPizzaStore = new NYPizzaStore();
```

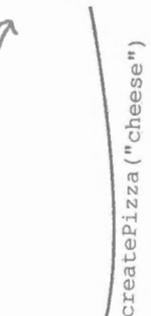
创建一个NYPizzaStore
的实例。



2 现在我们有了一家店，可以接订单了：

```
nyPizzaStore.orderPizza("cheese");
```

nyPizzaStore实例上的orderPizza()方法
被调用（在PizzaStore内定义的方法
行了）。



3 然后，orderPizza()方法调用createPizza()方法：

```
Pizza pizza = createPizza("cheese");
```

记住，工厂方法createPizza()在子类中实现。
在这个例子中，它返回一个纽约风味芝士比
萨。



4 最后，我们手上了还没准备的比萨，orderPizza()方法完成其准备：

```
pizza.prepare();  
pizza.bake();  
pizza.cut();  
pizza.box();
```

所有这些方法在工厂方法
createPizza()返回的特定比
萨中定义，而工厂方法在
NYPizzaStore中定义。

在不知道具体类是什么的
情况下，orderPizza()方法
返回一个Pizza。

刚刚忽略了一个东西：比萨！

没有比萨，我们的比萨店火不了，
我们来实现比萨



我们从一个抽象Pizza类开始，
所有具体比萨都派生自这个类。

```
public abstract class Pizza {  
    String name;  
    String dough;  
    String sauce;  
    List<String> toppings = new ArrayList<String>();
```

每个Pizza有一个名称、面团类型、
酱汁类型，以及一组馅料。

```
    void prepare() {  
        System.out.println("Preparing " + name);  
        System.out.println("Tossing dough...");  
        System.out.println("Adding sauce...");  
        System.out.println("Adding toppings: ");  
        for (String topping : toppings) {  
            System.out.println("    " + topping);  
        }  
    }
```

遵循许多有特定顺序的
步骤来准备。

```
    void bake() {  
        System.out.println("Bake for 25 minutes at 350");  
    }
```

抽象类为烘焙、切片和装盒
提供一些缺省的基本做法。

```
    void cut() {  
        System.out.println("Cutting the pizza into diagonal slices");  
    }
```

```
    void box() {  
        System.out.println("Place pizza in official PizzaStore box");  
    }
```

```
    public String getName() {  
        return name;  
    }  
}
```

记住：代码中我们不提供import和package语句。完整的源代码请从
wickedlysmart网站下载，地址：<https://wickedlysmart.com/head-first-design-patterns>

如果你搞丢了这个URL，可以在“概述”一节快速找到。

现在我们只需要一些具体子类……定义纽约和芝加哥风味的芝士比萨，如何？

```
public class NYStyleCheesePizza extends Pizza {

    public NYStyleCheesePizza() {
        name = "NY Style Sauce and Cheese Pizza";
        dough = "Thin Crust Dough";
        sauce = "Marinara Sauce";

        toppings.add("Grated Reggiano Cheese");
    }
}
```

纽约比萨有自己的大蒜番茄酱 (Marinara) 以及薄饼风味。

一种配料: Reggiano 芝士!

```
public class ChicagoStyleCheesePizza extends Pizza {

    public ChicagoStyleCheesePizza() {
        name = "Chicago Style Deep Dish Cheese Pizza";
        dough = "Extra Thick Crust Dough";
        sauce = "Plum Tomato Sauce";

        toppings.add("Shredded Mozzarella Cheese");
    }

    void cut() {
        System.out.println("Cutting the pizza into square slices");
    }
}
```

芝加哥比萨使用李子番茄作为酱汁，饼超厚。

芝加哥风味的深盘比萨有大量mozzarella芝士!

芝加哥风味比萨也覆盖cut()方法，使得切片是正方形。

你已经等得够久了。是时候来一些比萨了！

```
public class PizzaTestDrive {
```

```
    public static void main(String[] args) {
```

```
        PizzaStore nyStore = new NYPizzaStore();
```

```
        PizzaStore chicagoStore = new ChicagoPizzaStore();
```

```
        Pizza pizza = nyStore.orderPizza("cheese");
```

```
        System.out.println("Ethan ordered a " + pizza.getName() + "\n");
```

```
        pizza = chicagoStore.orderPizza("cheese");
```

```
        System.out.println("Joel ordered a " + pizza.getName() + "\n");
```

```
    }
```

```
}
```

首先我们创建两个不同的店。

我们用一个店来做Ethan的单……

……另一个做Joel的。

File Edit Window Help YouWantMootzOnThatPizza?

```
%java PizzaTestDrive
```

```
Preparing NY Style Sauce and Cheese Pizza
```

```
Tossing dough...
```

```
Adding sauce...
```

```
Adding toppings:
```

```
    Grated Reggiano cheese
```

```
Bake for 25 minutes at 350
```

```
Cutting the pizza into diagonal slices
```

```
Place pizza in official PizzaStore box
```

```
Ethan ordered a NY Style Sauce and Cheese Pizza
```

```
Preparing Chicago Style Deep Dish Cheese Pizza
```

```
Tossing dough...
```

```
Adding sauce...
```

```
Adding toppings:
```

```
    Shredded Mozzarella Cheese
```

```
Bake for 25 minutes at 350
```

```
Cutting the pizza into square slices
```

```
Place pizza in official PizzaStore box
```

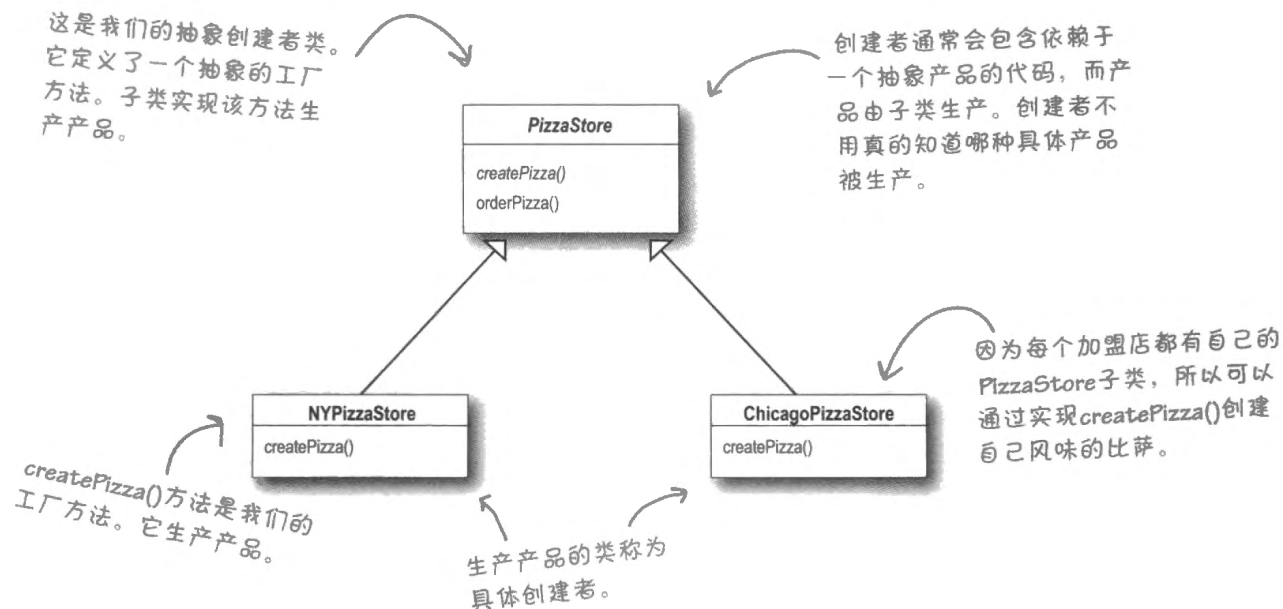
```
Joel ordered a Chicago Style Deep Dish Cheese Pizza
```

两个比萨都准备好了，馅料已加，已烘焙、切片、装盒。超类不必知道细节，子类只是通过实例化正确的比萨来处理所有这些。

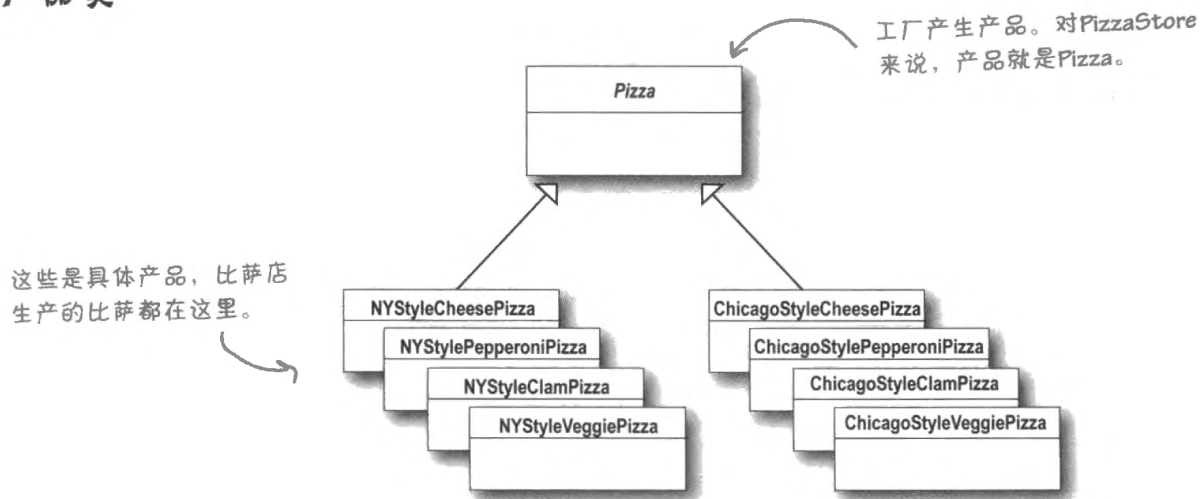
会见工厂方法模式的最后时刻到了

所有工厂模式都封装对象的创建。工厂方法模式通过让子类决定创建哪个对象，来封装对象的创建。我们来看看这些类图，了解这个模式中有哪些玩家：

创建者类



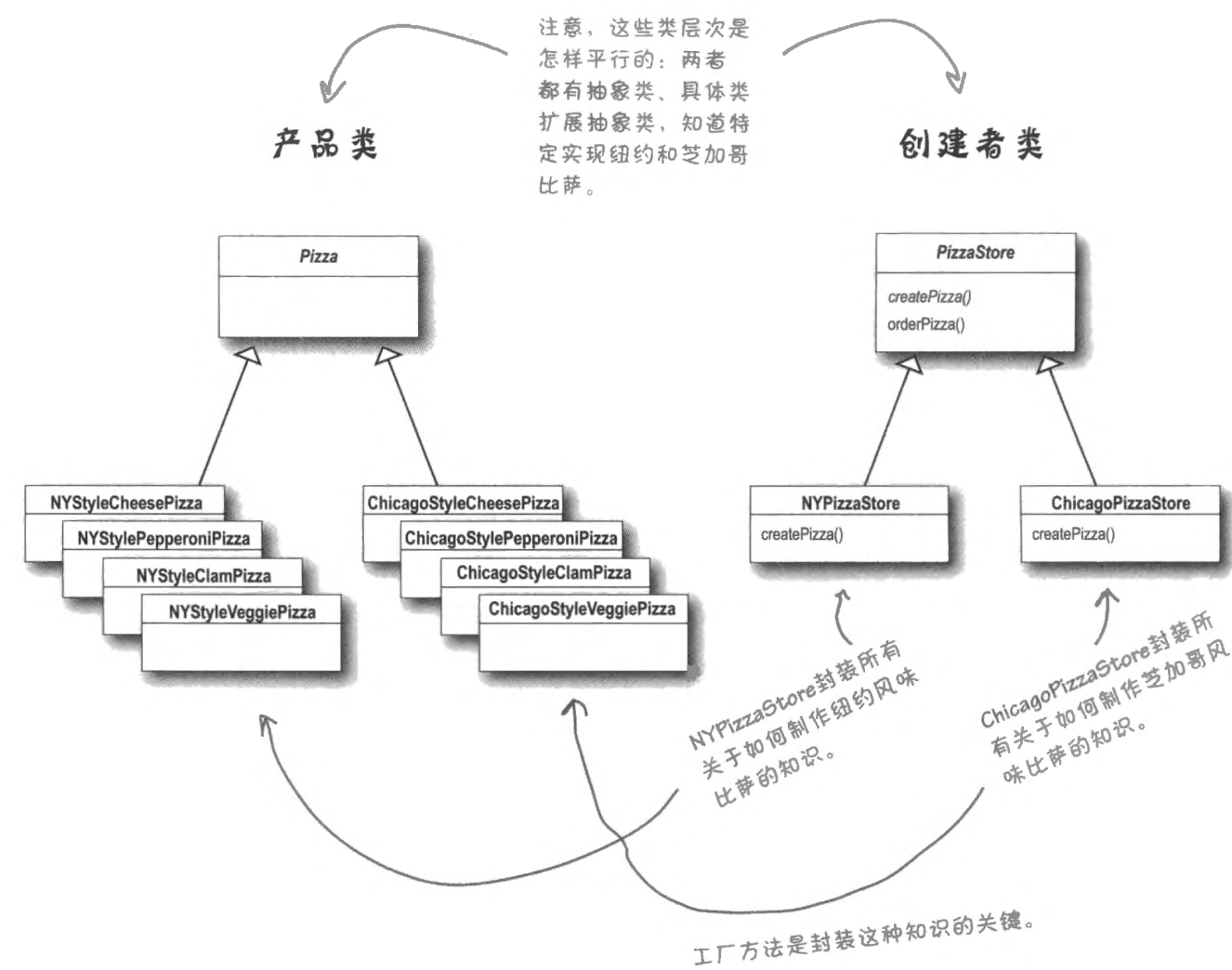
产品类



平行视角看创建者和产品

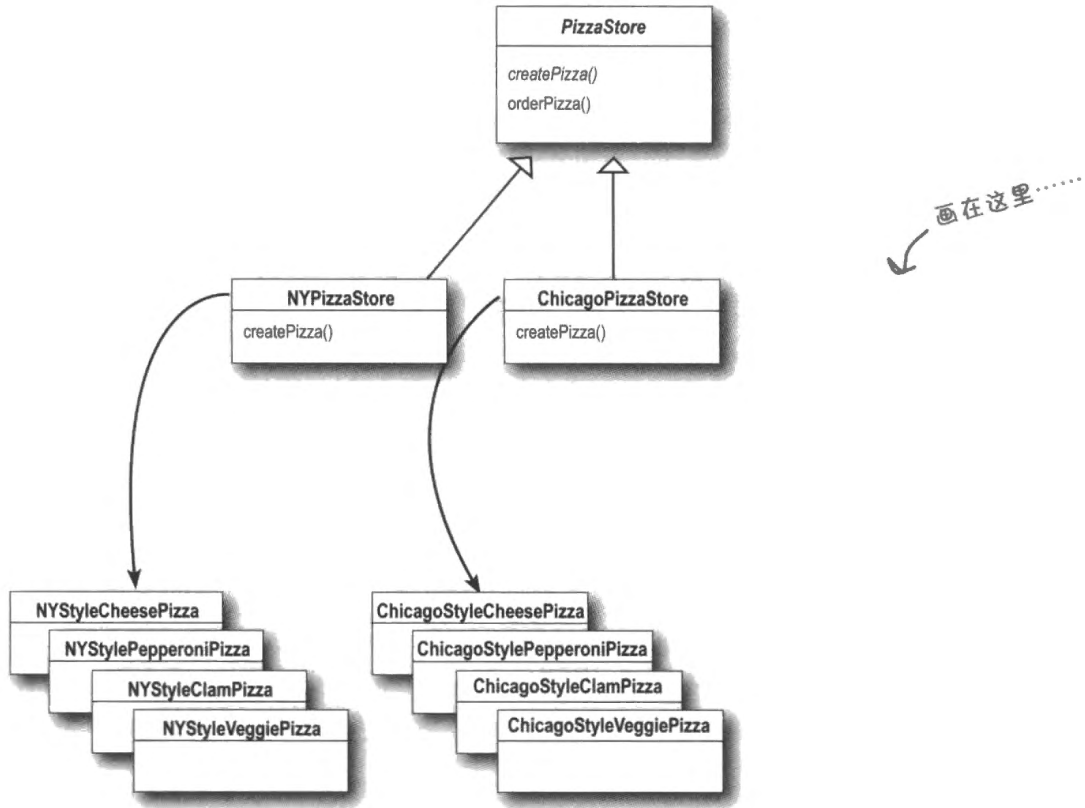
对于每一个具体创建者，通常有一整套产品要创建。芝加哥比萨创建者创建不同类型的芝加哥风味比萨，纽约比萨创建者创建不同类型的纽约风味比萨等。事实上，我们可以看到创建者类及其相应的产品类集是平行的层次。

我们来看看两个平行类层次以及它们如何相关：



设计谜题

我们需要另一种比萨，以符合疯狂的加州人的口味（当然，这里的疯狂是指好的那一方面）。请绘制出另一组平行的类，把加州区域纳入PizzaStore中。



OK，发挥你的想象力，找出五个“最奇特”的东西加入比萨中，写下来。
然后你就可以准备到加州去开比萨店了！

定义工厂方法模式

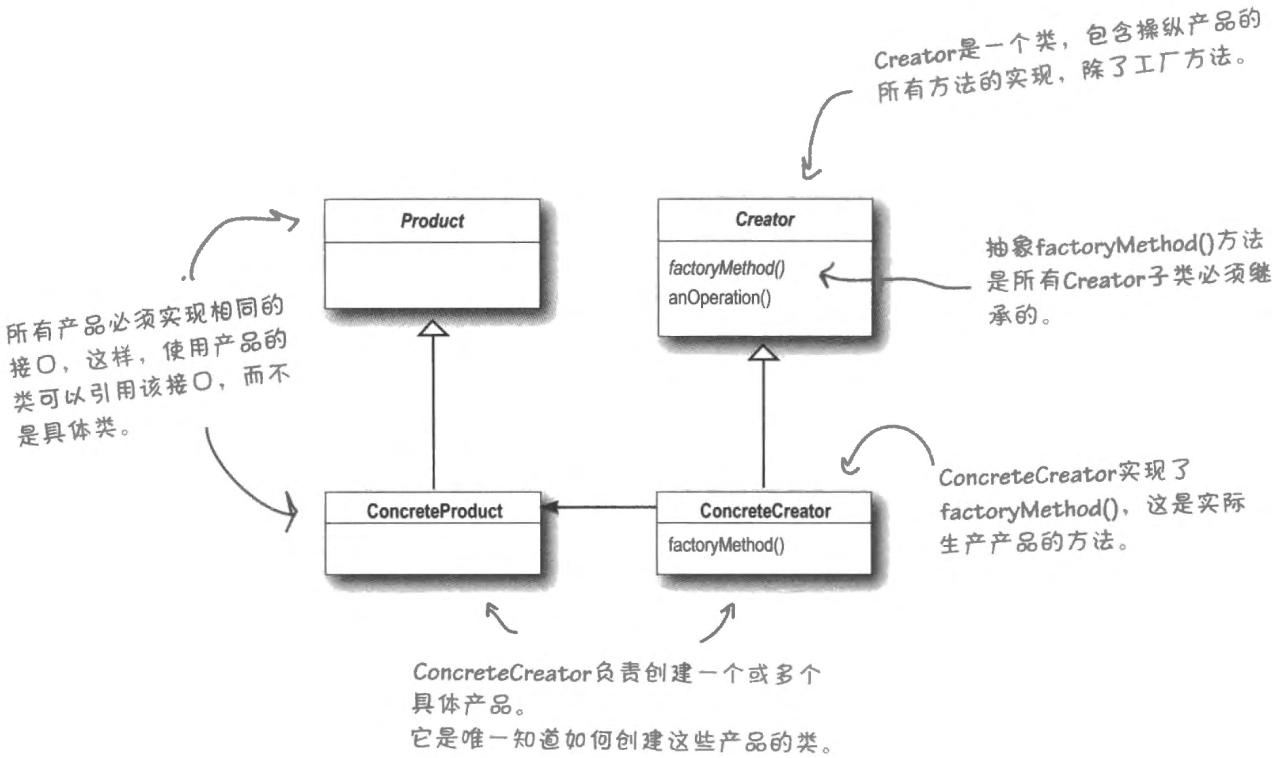
是时候拿出工厂方法模式的官方定义了：

工厂方法模式定义了一个创建对象的接口，但由子类决定要实例化哪个类。工厂方法让类把实例化推迟到子类。

通过每一个工厂，工厂方法模式让我们有办法封装具体类型的实例化。看看下面的类图，你可以看到，抽象的Creator类提供了一个带有创建对象方法（也称为“工厂方法”）的接口。在抽象的Creator中实现的任何其他的方法，是用来操作工厂方法生产的产品。只有子类真正实现工厂方法并创建产品。

正如官方定义中描述的，你会经常听到开发人员说，“工厂方法模式让子类决定实例化哪个类。”因为Creator类不知道将被创建的实际产品，我们说“决定”，不是因为该模式允许子类本身决定，该决定实际上指用哪个子类创建产品。

你可以问他们，“决定”是什么意思。我敢打赌，你比他们更清楚！



没有蠢问题

问：只有一个ConcreteCreator的时候，工厂方法模式有什么优点？

答：即使只有一个具体创建者，工厂方法模式依然有用，因为可以将产品的实现从其使用解耦。如果添加产品或者改变产品的实现，不会影响Creator（因为Creator没有和任何ConcreteProduct紧耦合）。

问：纽约和芝加哥的店是使用简单工厂实现的，这样的说法是否正确？看起来像这样。

答：它们类似，但用法不同。即使每个具体店的实现看起来像是SimplePizzaFactory，别忘了，这里的具体店扩展自一个类，这个类定义createPizza()为一个抽象方法。由每个店定义createPizza()方法的行为。在简单工厂中，工厂是另一个对象，和PizzaStore组合在一起。

问：工厂方法和Creator类总是抽象的吗？

答：不是，你可以定义一个缺省的工厂方法来生产一些具体产品。这样一来，你总是有办法创建产品，即使Creator类没有子类。

问：每个店可以基于传入的类型制作四种不同的披萨。是否所有的具体创建者都制作多种产品？还是有时只制作一种？

答：我们的实现称为“参数化工厂方法”。正如你注意到的，它可以根据传入的参数制作多于一种对象。但是，工厂经常只生产一种对象，不是参数化的。两者都是模式的有效形式。

问：你的参数化类型看起来不是“类型安全”的。我只是传入一个String！要是我请求一个“CalmPizza”（平静比萨）怎么办？

答：你无疑是正确的，这会导致所谓“运行时错误”。有若干其他更复杂的技巧可以用来使得参数更加“类型安全”——换句话说，确保参数中的错误在编译时被捕获。例如，你可以使用静态常量或者枚举创建代表参数类型的对象。

问：关于简单工厂和工厂方法之间的差异，我还是有点困惑。它们看起来很像，除了在工厂方法中，返回披萨的类是子类。可以解释一下吗？

答：你是对的。子类的确看起来很像简单工厂；但是，考虑到简单工厂是一锤子买卖，而工厂方法是创建一个框架，让子类决定使用哪个实现。例如，工厂方法模式中的orderPizza()方法提供了一个创建披萨的通用框架，依赖工厂方法实际创建制作披萨的具体类。通过子类化PizzaStore类，你决定哪一个具体产品来制作orderPizza()返回的披萨。和简单工厂相比，简单工厂只是提供封装对象创建的一种方式，但没有提供工厂方法的弹性，因为它没有办法改变正在创建的产品。



大师和徒弟……

大师：告诉我训练进行得如何了？

徒弟：大师，我已经进一步研究了“封装变化”。

大师：继续……

徒弟：我已经学习到可以封装创建对象的代码。实例化具体类的代码，是一个频繁变化的区域。我学到一个称为“工厂”的技巧，可以封装这个实例化的行为。

大师：这些“工厂”有什么好处？

徒弟：好处很多。将所有创建代码放进一个对象或方法中，可以避免代码中的重复，并为维护提供了空间。这也意味着客户在实例化对象时，只依赖于接口，而不是具体类。正如我在学习中发现的，这允许我针对接口编程，而不针对实现编程，让我的代码更有弹性，未来更有扩展性。

大师：是的，你的OO本能在成长。今天有问题要问吗？

徒弟：大师，我知道，通过封装对象创建，我在对抽象编码，并将客户代码从真实的实现解耦。但我的工厂代码依然必须使用具体类来实例化真正的对象。我这不是自欺欺人吗？

大师：对象创建是生活的现实，我们必须创建对象，否则就连一个简单的Java应用也创建不了。但是，有了这个现实的知识，我们可以设计我们的代码，把创建代码围起来，就像把羊毛堆到你面前看着一样。一旦围起来，我们可以保护和照顾创建代码。如果我们让创建代码到处乱跑，就不可能收集“羊毛”了。

徒弟：大师，我从中看到了真理。

大师：我知道你会的。现在，请继续思考对象依赖。



削尖你的铅笔

假装你从未听说过OO工厂。下面是一个“依赖性很强”、没有使用工厂的PizzaStore版本。我们需要你来数一数，这个类所依赖的具体披萨类的数目。如果添加加州风味披萨到这个PizzaStore，又会依赖多少个类？

```
public class DependentPizzaStore {

    public Pizza createPizza(String style, String type) {
        Pizza pizza = null;
        if (style.equals("NY")) {
            if (type.equals("cheese")) {
                pizza = new NYStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new NYStyleVeggiePizza();
            } else if (type.equals("clam")) {
                pizza = new NYStyleClamPizza();
            } else if (type.equals("pepperoni")) {
                pizza = new NYStylePepperoniPizza();
            }
        } else if (style.equals("Chicago")) {
            if (type.equals("cheese")) {
                pizza = new ChicagoStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new ChicagoStyleVeggiePizza();
            } else if (type.equals("clam")) {
                pizza = new ChicagoStyleClamPizza();
            } else if (type.equals("pepperoni")) {
                pizza = new ChicagoStylePepperoniPizza();
            }
        } else {
            System.out.println("Error: invalid type of pizza");
            return null;
        }
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }
}
```

处理所有纽约风味
披萨

处理所有芝加哥风味
披萨

你可以把答案写在
这里：

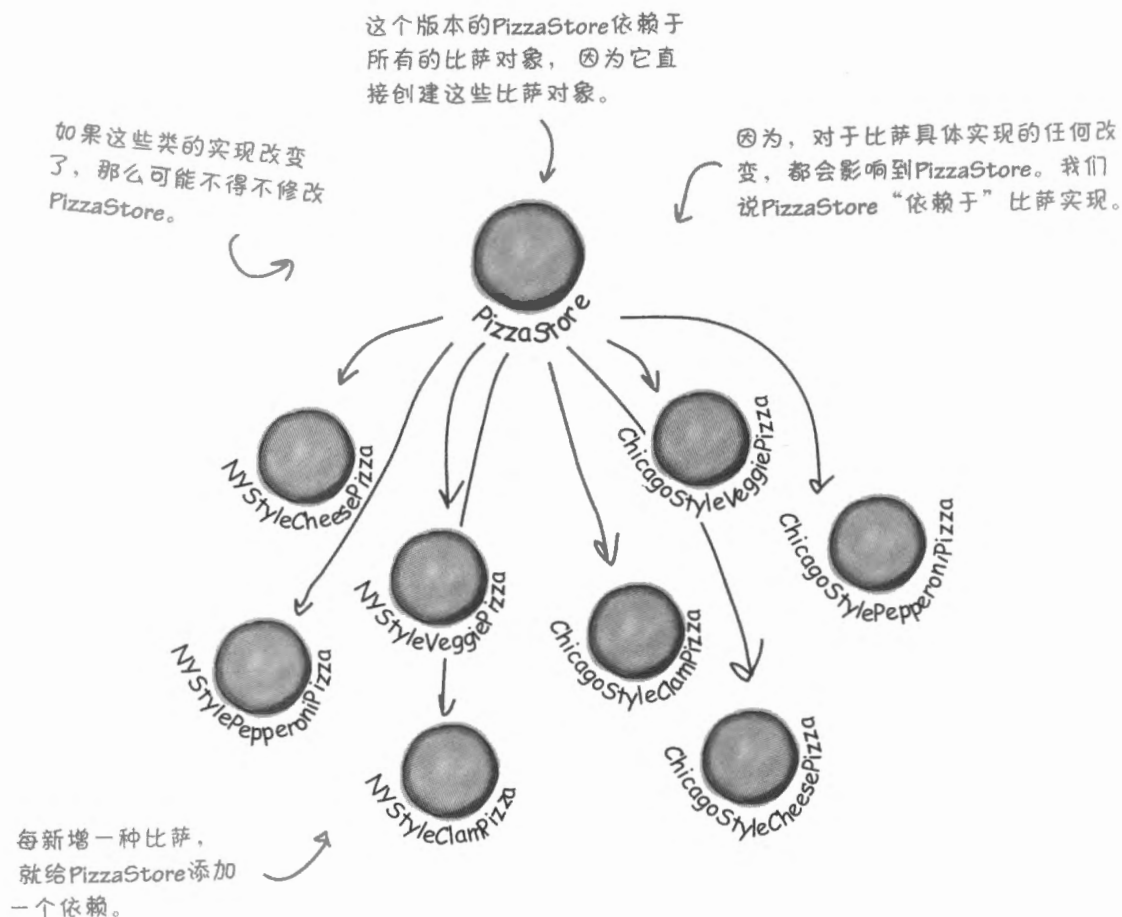
数目

加上加州风味后的
数目

看看对象依赖

当你直接实例化一个对象时，你在依赖于其具体类。看一看上一节“非常依赖”的PizzaStore。它就在PizzaStore类中创建所有比萨对象，而不是委托给工厂。

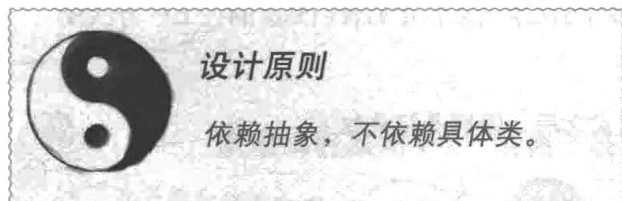
如果我们画图表达该PizzaStore版本以及它依赖的所有对象，看起来像这样：



依赖倒置原则

很显然，在我们的代码中减少对具体类的依赖是一件“好事”。事实上，有一个OO设计原则正式阐明了这一点；这个原则甚至还有一个响亮又正式的名称：依赖倒置原则（Dependency Inversion Principle）。

通用原则如下：



又是一个你可以用来让主管对你印象深刻的词语！你所获得的加薪，将比买本书所付出的钱更多，而且还会赢得开发小伙伴的钦佩。

首先，这个原则听起来很像“针对接口编程，不针对实现编程”，对吧？是很像，但是，依赖倒置原则更强调抽象。该原则说明，高层组件不应该依赖于低层组件，而且，它们都应该依赖于抽象。

但这到底是什么意思？

这个嘛，我们先再来看看上一页比萨店的图。PizzaStore是我们的“高层组件”，比萨实现是“低层组件”，显然，PizzaStore依赖于具体比萨类。

“高层”组件是一个类，其行为以其他“低层”组件的形式定义。

例如，PizzaStore是个高层组件，因为它的行为以比萨的形式定义：PizzaStore创建所有不同的比萨对象、准备、烘焙、切片、装盒；而所用的比萨是低层组件。

现在，这个原则告诉我们，应该重新编写我们的代码，以便依赖于抽象，而不是具体类。这对高层模块和低层模块都适用。

但是怎么做？我们来考虑如何把这个原则应用到我们“非常依赖”的PizzaStore实现……

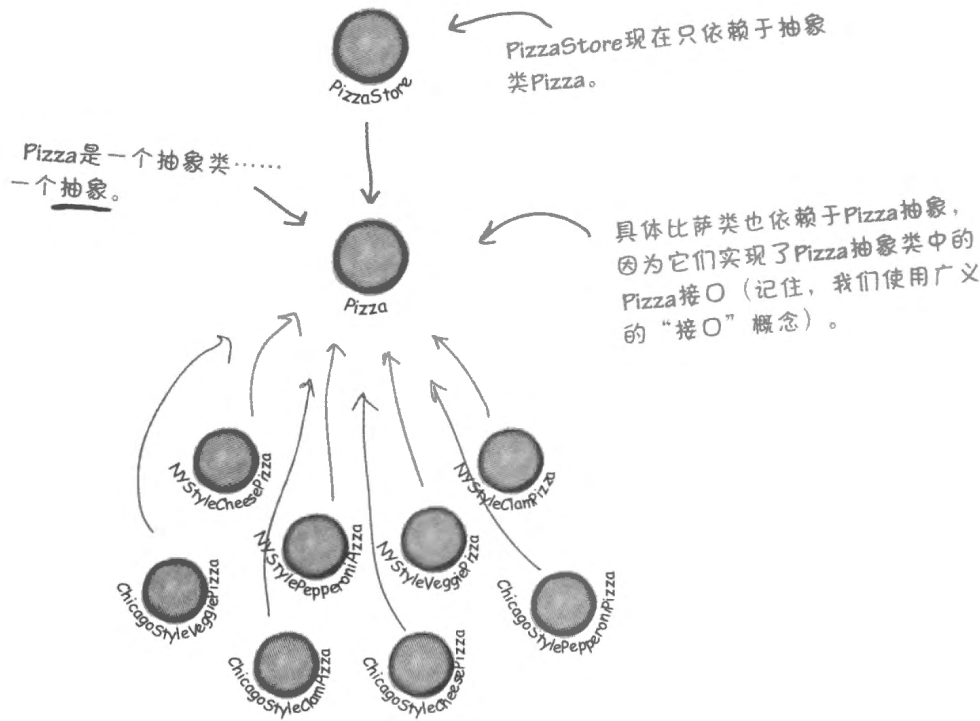
应用原则

现在，“非常依赖”版PizzaStore的主要问题是，它依赖于每个比萨类型，因为实际上它是在自己的orderPizza()方法中，实例化具体类型。

虽然已经创建了一个抽象，即Pizza，不过我们是在代码中创建具体的Pizza，因此，这个抽象没有带来太多好处。

怎样将这些实例化从orderPizza()方法拿出来？正如我们知道的，工厂方法模式正好允许我们做到这一点。

因此，在我们应用工厂方法模式之后，图形看起来如下：



应用工厂方法之后，你会注意到，高层组件PizzaStore以及低层组件（也就是这些比萨）都依赖于抽象，Pizza。工厂方法不是唯一遵循依赖倒置原则的技巧，但它是最有威力的一个。



OK, 我知道依赖是什么了, 但为什么叫作“依赖倒置”?

依赖倒置原则中, “倒置”在哪?

依赖倒置原则名称中的“倒置”, 是因为它倒转了通常考虑OO设计的方式。你看看上一页的图。注意, 低层组件现在依赖于更高层的抽象。同样, 高层组件也绑定到同一抽象。因此, 几页前我们画的自上而下的依赖图自己倒转过来了, 高层和低层模块现在都依赖于抽象。

我们也来过一遍典型设计过程背后的思想, 了解引进该原则如何能够倒置我们考虑设计的方式……

倒置你的思考……



嗯，比萨店准备、烘焙、装盒，因此我的商店需要能制作许多不同的比萨。例如：芝士比萨、素食比萨、蛤蜊比萨等……



好吧，芝士比萨、素食比萨和蛤蜊比萨都是比萨，因此它们应该共享一个**Pizza**接口。



既然现在有一个**Pizza**抽象，我可以在不用操心具体比萨类的情况下设计比萨店。

OK，因此你需要实现一个比萨店。你想到的第一件事情是什么？

没错，先从顶端开始，然后往下到具体类。但是，正如你已经看到的，你不想要你的比萨店知道具体的比萨类型，因为会导致依赖于所有这些具体类！

不从顶端开始，而是从那些**Pizza**开始，思考你可以抽象什么。

对！你在考虑**Pizza**抽象。所以，现在回头再次思考比萨店的设计。

接近了。但是这么做的话，你不得不依靠一个工厂将这些具体类从比萨店取出。一旦你这么做了，你的各种不同具体比萨类型只依赖于一个抽象，比萨店也依赖这个抽象。随着你的思考，针对一个商店依赖于具体类的设计，我们倒置了其中的依赖。

帮助你遵循该原则的几条指南……

以下指南可以帮助你避免OO设计违反依赖倒置原则：

- 变量不应该持有到具体类的引用。

如果使用new，就会持有到具体类的引用。通过使用工厂来绕开！

- 类不应该派生自具体类。

如果派生自具体类，就会依赖具体类。派生自一个抽象（接口或抽象类）。

- 方法不应该覆盖其任何基类的已实现方法。

如果覆盖已实现的方法，那么基类就不是一个真正适合被继承的抽象。基类中这些已实现的方法，应该由所有的子类共享。

但是，等一下，遵循这些指南不太可能吧？如果我遵循这些指南，我连一个简单的程序都写不出来！

我们都很清楚，任何Java程序都有违反这些指南的地方！

你说的没错！像许多原则一样，这个指南你应该尽量遵循，但不要当成任何时候都应该遵循的铁律。显然，写出来的每个Java程序都有违反这些指南的地方！

但是，如果你把这些指南融会贯通，在设计时藏在大脑深处，当违反原则时，你会知道在违反原则，并且会有一个好的理由。例如，如果你知道有一个类不可能变化，那么，在你的代码中实例化一个具体类也不是世界末日。想一想，我们一直不假思索地实例化String对象，违反这个原则了吗？有。可以这么做吗？可以！为什么？因为String不可能改变。

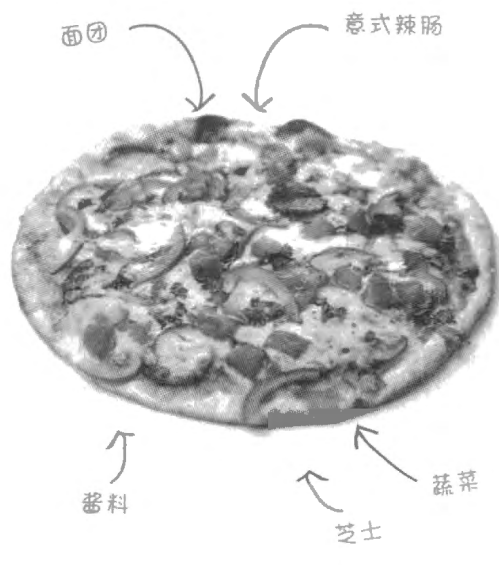
另一方面，如果一个类有可能变化，可以用一些良好的技巧，像工厂方法，来封装变化。



与此同时，回到比萨店……

比萨店的设计真的被塑造得很棒：有弹性的框架，而且遵循设计原则。

现在，对象村比萨成功的关键一直在于新鲜、高质量的原料，而且你发现，有了新的框架，加盟店遵循了你的流程，但是有一些加盟店在他们的比萨中用劣质原料以降低成本和增加利润。你知道必须采取一些手段，因为长此以往会毁了对象村的品牌。



确保原料的一致

如何确保每家加盟店使用高质量的原料？你打算建造一家生产原料的工厂，并将原料配送到各家加盟店！

对于这个计划，现在只有一个问题：加盟店坐落在不同的区域，纽约的红酱料和芝加哥的红酱料是不一样的。因此，你有一组需要配送到纽约的原料，另一组不同的原料配送到芝加哥。我们来看得更仔细一点：



芝加哥 比萨菜单

- 芝士比萨**
李子番茄酱、Mozzarella芝士、Parmesan芝士、牛至
- 素食比萨**
李子番茄酱、Mozzarella芝士、Parmesan芝士、茄子、菠菜、黑橄榄
- 蛤蜊比萨**
李子番茄酱、Mozzarella芝士、Parmesan芝士、蛤蜊
- 意式辣肠比萨**
李子番茄酱、Mozzarella芝士、Parmesan芝士、意式辣肠

我们有相同的产品家族（面团、意式辣肠、酱料、芝士、蔬菜、肉），但根据区域有不同实现。

纽约 比萨菜单

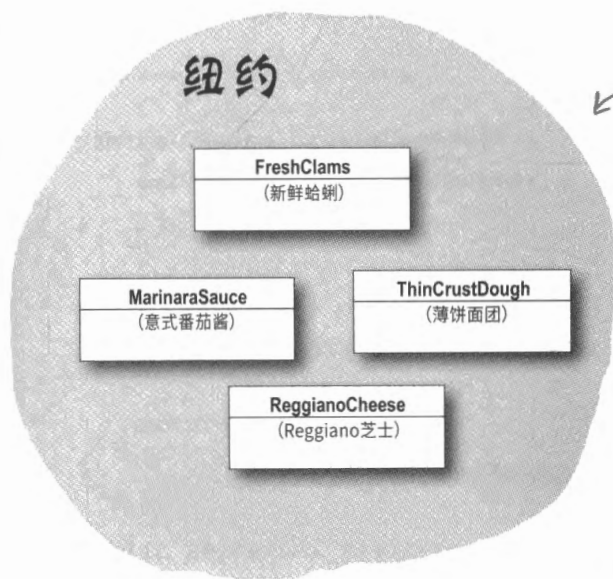


- 芝士比萨**
意式番茄酱、Reggiano芝士、蒜
- 素食比萨**
意式番茄酱、Reggiano芝士、蘑菇、洋葱、红椒
- 蛤蜊比萨**
意式番茄酱、Reggiano芝士、新鲜蛤蜊
- 意式辣肠比萨**
意式番茄酱、Reggiano芝士、蘑菇、洋葱、红椒、意式辣肠

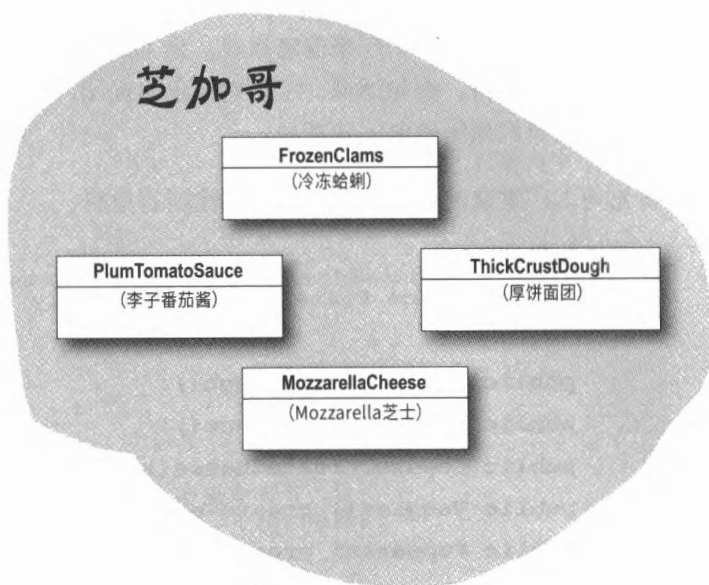
原料家族……

纽约使用一组原料，而芝加哥使用另一组。对象村比萨是如此受欢迎，看来不用太长时间，你又要给加州配送另一组区域原料了，接下来是？奥斯汀？

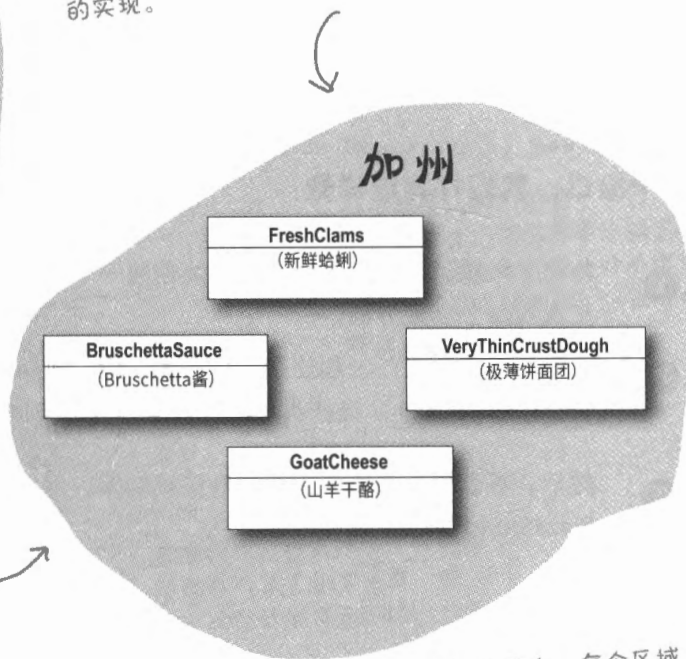
要想搞定这个，你不得不弄清楚如何处理原料家族。



每个家族都包含了一种面团、一种酱料、一种芝士、以及一种海鲜馅料（以及一些没写出来的原料，例如蔬菜与香料）。



对象村的所有比萨都用相同的组件制造，但是每个区域对于这些组件有不同的实现。



总之，这三个区域组成了原料家族的集合，每个区域实现了一个完整的原料家族。

建造原料工厂

现在，我们打算建造一个工厂来创建原料。该工厂将负责创建原料家族中的每一个原料。换句话说，工厂需要创建面团、酱、芝士等。你马上会看到我们如何处理区域差异。

我们首先为工厂定义一个接口，该接口创建所有原料：

```
public interface PizzaIngredientFactory {
```

```
    public Dough createDough();  
    public Sauce createSauce();  
    public Cheese createCheese();  
    public Veggies[] createVeggies();  
    public Pepperoni createPepperoni();  
    public Clams createClam();
```

```
}
```

这里有大量新类，每个原料一个。

在接口中，对于每个原料我们都定义一个创建方法。

有了这个接口，我们打算这样做：

- 1 为每个区域建造一个工厂。你需要创建一个PizzaIngredientFactory的子类来实现每个创建方法。
- 2 实现一组要和工厂一起使用的原料类，像ReggianoCheese、RedPeppers和ThickCrustDough。这些类可以在合适的区域之间共享。
- 3 然后，我们依然需要把所有这些连接起来，把新的原料工厂整合进老的PizzaStore代码。

建造纽约原料工厂

OK，以下是纽约原料工厂的实现。这个工厂专门生产意式番茄酱、Reggiano芝士、新鲜蛤蜊等。

所有原料工厂都实现这个接口，
包括纽约原料工厂。

```
public class NYPizzaIngredientFactory implements PizzaIngredientFactory {

    public Dough createDough() {
        return new ThinCrustDough();
    }

    public Sauce createSauce() {
        return new MarinaraSauce();
    }

    public Cheese createCheese() {
        return new ReggianoCheese();
    }

    public Veggies[] createVeggies() {
        Veggies veggies[] = { new Garlic(), new Onion(), new Mushroom(), new RedPepper() };
        return veggies;
    }

    public Pepperoni createPepperoni() {
        return new SlicedPepperoni();
    }

    public Clams createClam() {
        return new FreshClams();
    }
}
```

对原料家族中的每个原料，
我们都创建了纽约版本。

对于蔬菜，我们返回一个蔬菜数组。
在这里，我们把蔬菜硬编码了。其实
可以写得更精致，但这对于学习工厂
模式没什么增值作用，因此我们保持
简单。

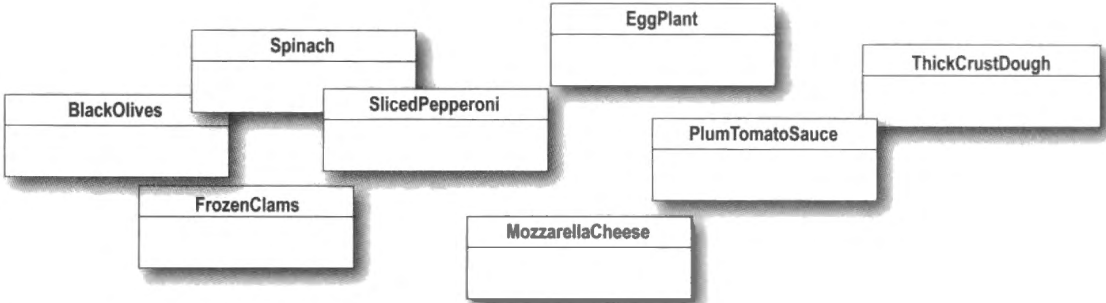
最棒的切片意式辣肠，纽约和
芝加哥共享它。下一页，你
自己实现芝加哥工厂时，务必
使用它。

纽约靠海，有新鲜蛤蜊。
芝加哥只能用冷冻蛤蜊。



削尖你的铅笔

编写ChicagoPizzaIngredientFactory。你可以在实现中引用以下类：



重做比萨……

工厂已经一切就绪，要准备生产高质量原料了。现在，我们只需要重做我们的比萨，让它只使用工厂生产的原料。先从抽象Pizza类开始：

```
public abstract class Pizza {
    String name;

    Dough dough;
    Sauce sauce;
    Veggies veggies[];
    Cheese cheese;
    Pepperoni pepperoni;
    Clams clam;

    abstract void prepare();

    void bake() {
        System.out.println("Bake for 25 minutes at 350");
    }

    void cut() {
        System.out.println("Cutting the pizza into diagonal slices");
    }

    void box() {
        System.out.println("Place pizza in official PizzaStore box");
    }

    void setName(String name) {
        this.name = name;
    }

    String getName() {
        return name;
    }

    public String toString() {
        // 这里是打印比萨的代码
    }
}
```

每个比萨持有一组原料，在准备时会用到。

现在把prepare()方法声明成抽象的。这个方法收集比萨所需的原料，当然，这些原料来自原料工厂。

其他的方法保持不变，除了prepare方法。

重做比萨，继续……

现在，你有了抽象的Pizza类，是时候创建纽约和芝加哥风味比萨了。从今以后，加盟店将直接从工厂取得原料，偷工减料的日子结束了！

当我们编写工厂方法代码时，有过NYCheesePizza和ChicagoCheesePizza类。如果你观察这两个类，会发现唯一的差别在于使用区域原料。比萨的做法是一样的（面团+ 酱料+ 芝士），其他的比萨（蔬菜、蛤蜊等）也是如此。它们都遵循相同的准备步骤，只是使用不同的原料。

因此，你将看到的是，我们真的不需要为每个比萨设计两个类；原料工厂为我们处理区域差异。

CheesePizza如下：

```
public class CheesePizza extends Pizza {
    PizzaIngredientFactory ingredientFactory;

    public CheesePizza(PizzaIngredientFactory ingredientFactory) {
        this.ingredientFactory = ingredientFactory;
    }

    void prepare() {
        System.out.println("Preparing " + name);
        dough = ingredientFactory.createDough();
        sauce = ingredientFactory.createSauce();
        cheese = ingredientFactory.createCheese();
    }
}
```

现在要制作比萨，我们需要一个工厂来提供原料。因此，每个Pizza类在其构造器中会有一个工厂传入，并把它存储在一个实例变量中。

← 魔法在这里！

prepare()方法一步一步地创建芝士比萨，每次需要原料时，就请求工厂生产。



Code up Close

Pizza的代码用所组合的工厂生产比萨所用的原料。所产生的原料依赖所用的工厂。Pizza类不关心，它知道如何制作比萨。现在，Pizza从区域原料差异解耦，无论工厂是在奥斯汀、纳什维尔还是其他地方，Pizza类可以轻易地复用。

```
sauce = ingredientFactory.createSauce();
```

设置Pizza的实例变量，引用到这个比萨所用的特定酱料。

这是原料工厂。Pizza类不在意用什么工厂，只要是原料工厂就行。

createSauce()方法返回其区域所用的酱料。如果这是一个纽约原料工厂，得到的是意式番茄酱。

我们也来看看ClamPizza：

```
public class ClamPizza extends Pizza {
    PizzaIngredientFactory ingredientFactory;
```

ClamPizza也藏了一个原料工厂。

```
    public ClamPizza(PizzaIngredientFactory ingredientFactory)
```

```
{
```

```
        this.ingredientFactory = ingredientFactory;
```

```
}
```

```
void prepare() {
```

```
    System.out.println("Preparing " + name);
```

```
    dough = ingredientFactory.createDough();
```

```
    sauce = ingredientFactory.createSauce();
```

```
    cheese = ingredientFactory.createCheese();
```

```
    clam = ingredientFactory.createClam();
```

```
}
```

```
}
```

为了做蛤蜊比萨，prepare()方法从本地工厂中收集正确的原料。

如果是纽约工厂，蛤蜊是新鲜的；如果是芝加哥工厂，蛤蜊是冷冻的。

重游比萨店

我们快完工了，只需要快速巡视一下加盟店，以确认它们在使用正确的Pizza，也需要给它们一个本地原料工厂的引用：

```
public class NYPizzaStore extends PizzaStore {  
  
    protected Pizza createPizza(String item) {  
        Pizza pizza = null;  
        PizzaIngredientFactory ingredientFactory =  
            new NYPizzaIngredientFactory();  
  
        if (item.equals("cheese")) {  
  
            pizza = new CheesePizza(ingredientFactory);  
            pizza.setName("New York Style Cheese Pizza");  
  
        } else if (item.equals("veggie")) {  
  
            pizza = new VeggiePizza(ingredientFactory);  
            pizza.setName("New York Style Veggie Pizza");  
  
        } else if (item.equals("clam")) {  
  
            pizza = new ClamPizza(ingredientFactory);  
            pizza.setName("New York Style Clam Pizza");  
  
        } else if (item.equals("pepperoni")) {  
  
            pizza = new PepperoniPizza(ingredientFactory);  
            pizza.setName("New York Style Pepperoni Pizza");  
  
        }  
        return pizza;  
    }  
}
```

纽约店组合一个纽约比萨原料工厂，该原料工厂用于生产所有纽约风味比萨所需的原料。

我们现在把用来生产原料的工厂传递给每一个比萨。

退回上一页，确认你理解了比萨和工厂是如何合作的！

对于每一类比萨，实例化一个新的Pizza，并给它所需的工厂以便取得原料。



把这个版本的createPizza()方法和本章前面的工厂方法实现比较一下。

我们做了什么？

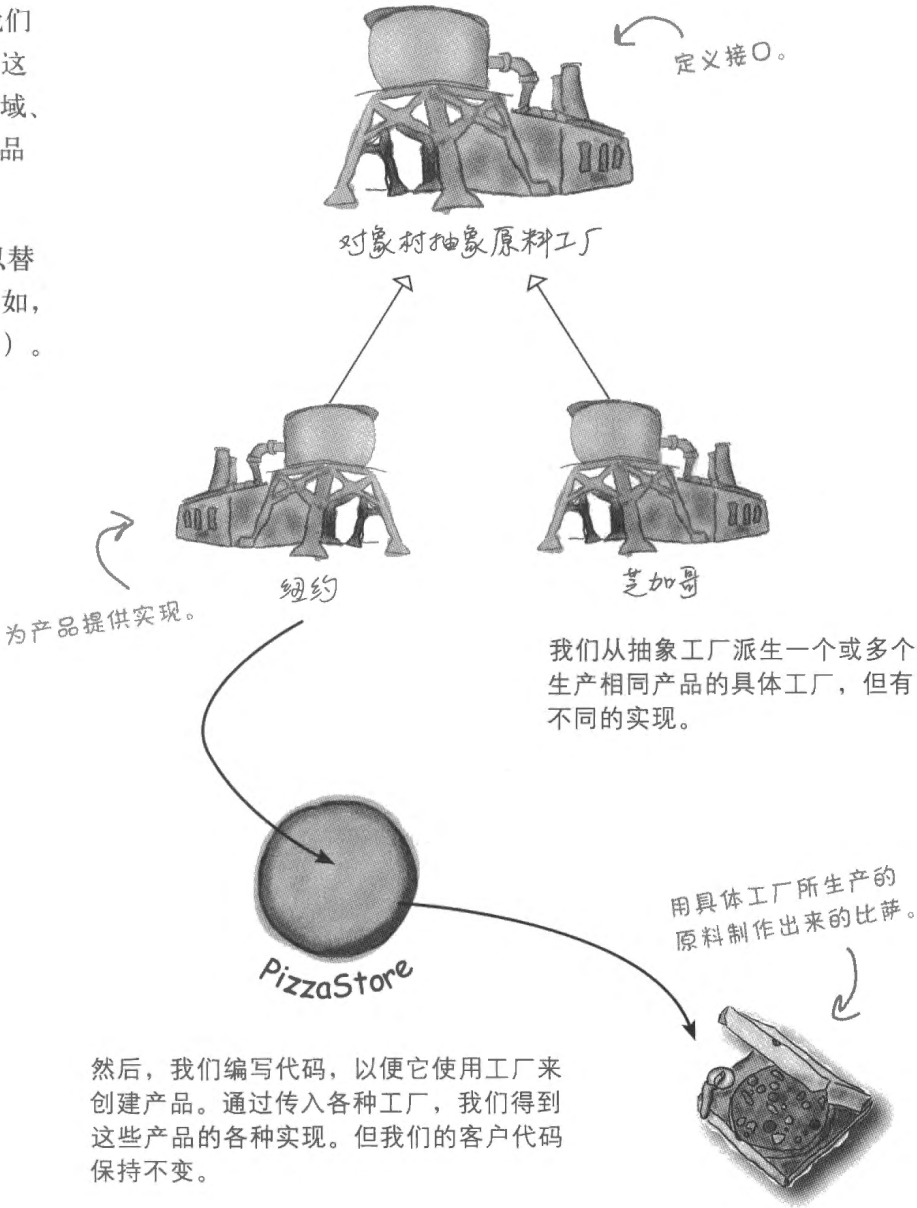
一连串的代码变化，我们到底做了什么？

通过引进一个称为抽象工厂的新工厂类型，我们提供了为比萨创建原料家族的方法。

抽象工厂给我们一个创建产品家族的接口。通过使用这个接口编写代码，我们把代码从实际创建产品的工厂解耦。这让我们在为不同上下文（例如不同区域、不同操作系统或者不同视感）生产产品时，能够实现工厂的变化。

因为代码从实际产品解耦，我们可以替换不同的工厂来获得不同的行为（例如，获得意式番茄酱，而不是李子番茄酱）。

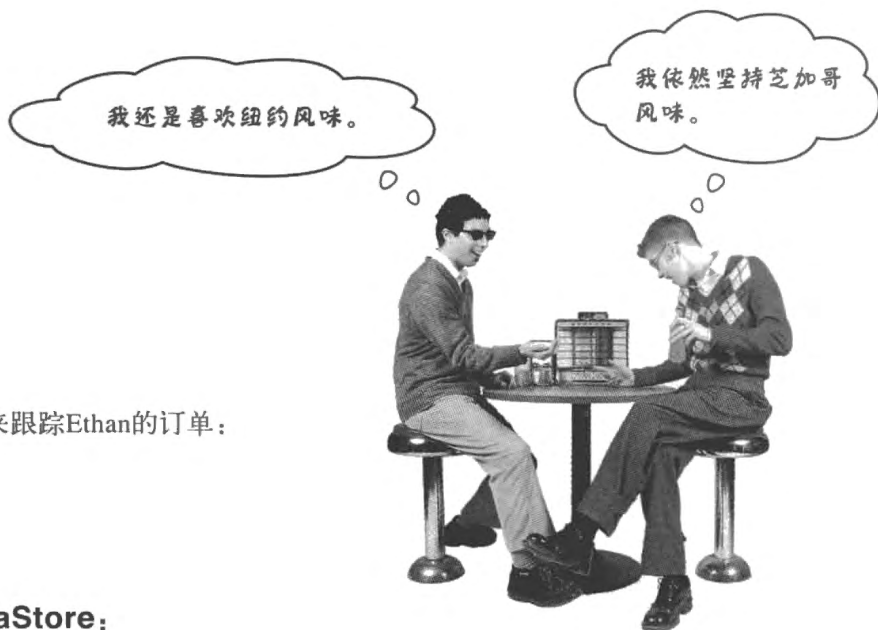
抽象工厂为产品家族提供一个接口。家族是什么？在我们的例子中，是我们做比萨需要的所有东西：面团、酱料、芝士、肉以及蔬菜。



给Ethan和Joel更多的比萨……

Ethan和Joel对对象村的比萨百吃不厌！他们并不知道，现在所订购的比萨是用新原料工厂做的。因此，现在当他们下单时……

幕后花絮



订购流程的第一部分根本没变，我们再来跟踪Ethan的订单：

1 首先，我们需要一个NYPizzaStore：

```
PizzaStore nyPizzaStore = new NYPizzaStore();
```

创建一个NYPizzaStore
的实例。

2 有了一个店，我们可以下单了：

```
nyPizzaStore.orderPizza("cheese");
```

nyPizzaStore实例上的orderPizza()
方法被调用。

3 orderPizza()方法首先调用createPizza()方法：

```
Pizza pizza = createPizza("cheese");
```

createPizza("cheese")

(见下页。)

从这开始有变化，因为我们使用了一个原料工厂

幕后花絮

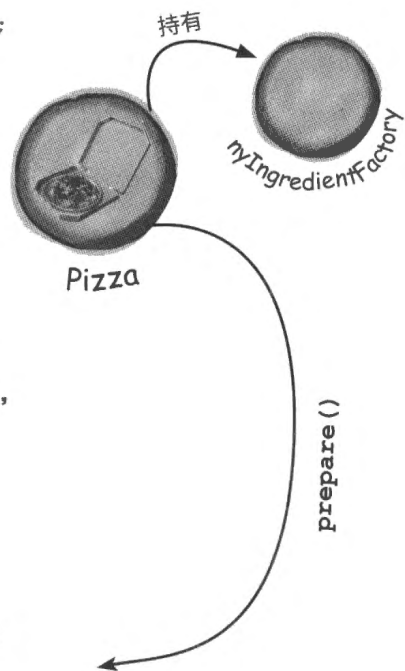


4 当createPizza()方法被调用，原料工厂介入：

原料工厂被选中，并在PizzaStore中实例化，然后被传入每个披萨的构造器。

```
Pizza pizza = new CheesePizza(nyIngredientFactory);
```

创建一个组合纽约原料工厂的Pizza实例。



5 接下来，我们需要准备比萨。一旦prepare()方法被调用，工厂被要求准备原料：

```
void prepare() {
    dough = factory.createDough();
    sauce = factory.createSauce();
    cheese = factory.createCheese();
}
```

薄饼

意式番茄酱

Reggiano芝士

Ethan的比萨用了纽约原料工厂，因此我们获得纽约的原料。

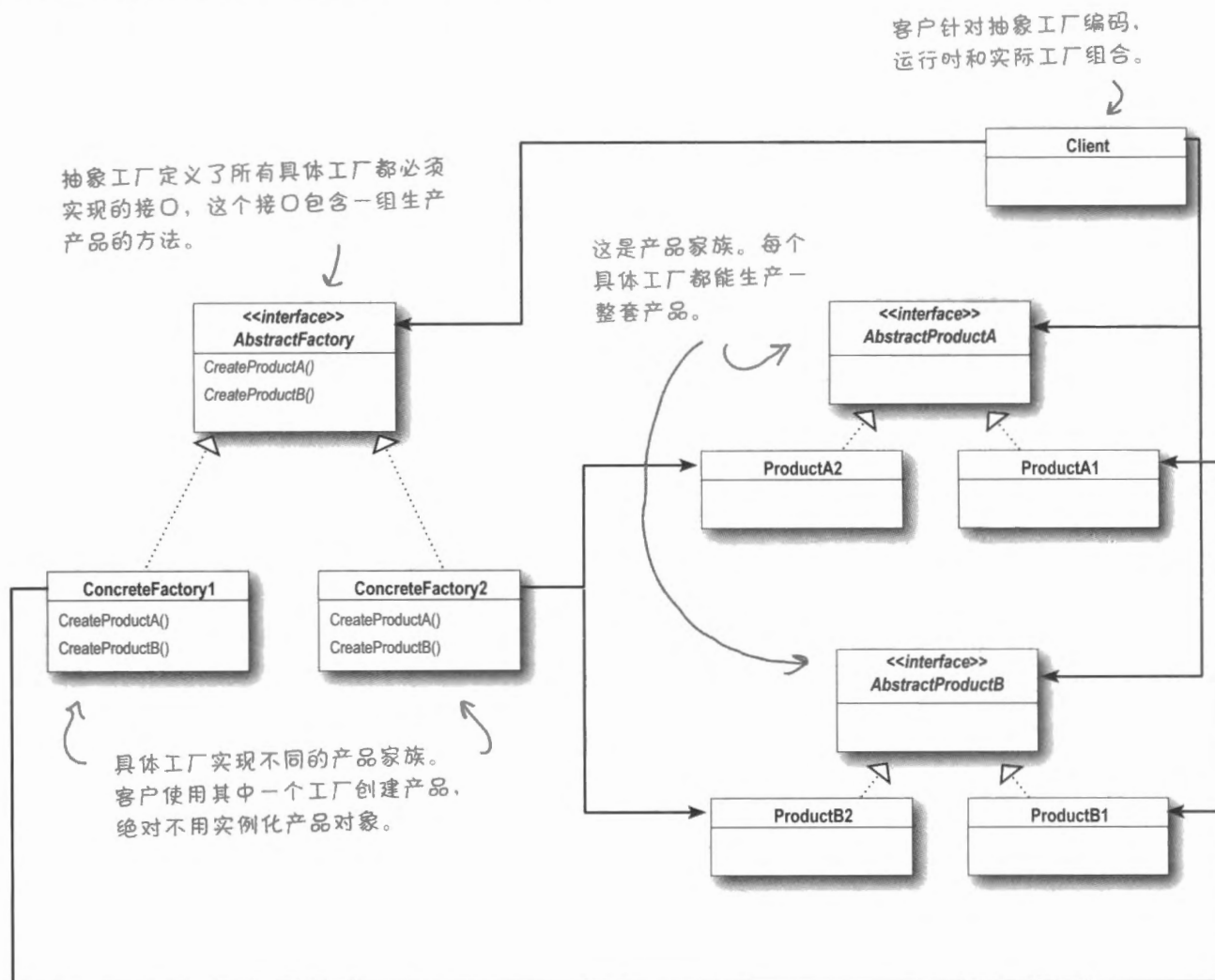
6 最后，有了准备好的比萨，orderPizza()就会烘焙、切片，以及装盒。

定义抽象工厂模式

我们又给模式家族添加了另一个工厂模式，这个模式可以创建产品的家族。我们来看看这个模式的官方定义：

抽象工厂模式提供一个接口来创建相关或依赖对象的家族，而不需要指定具体类。

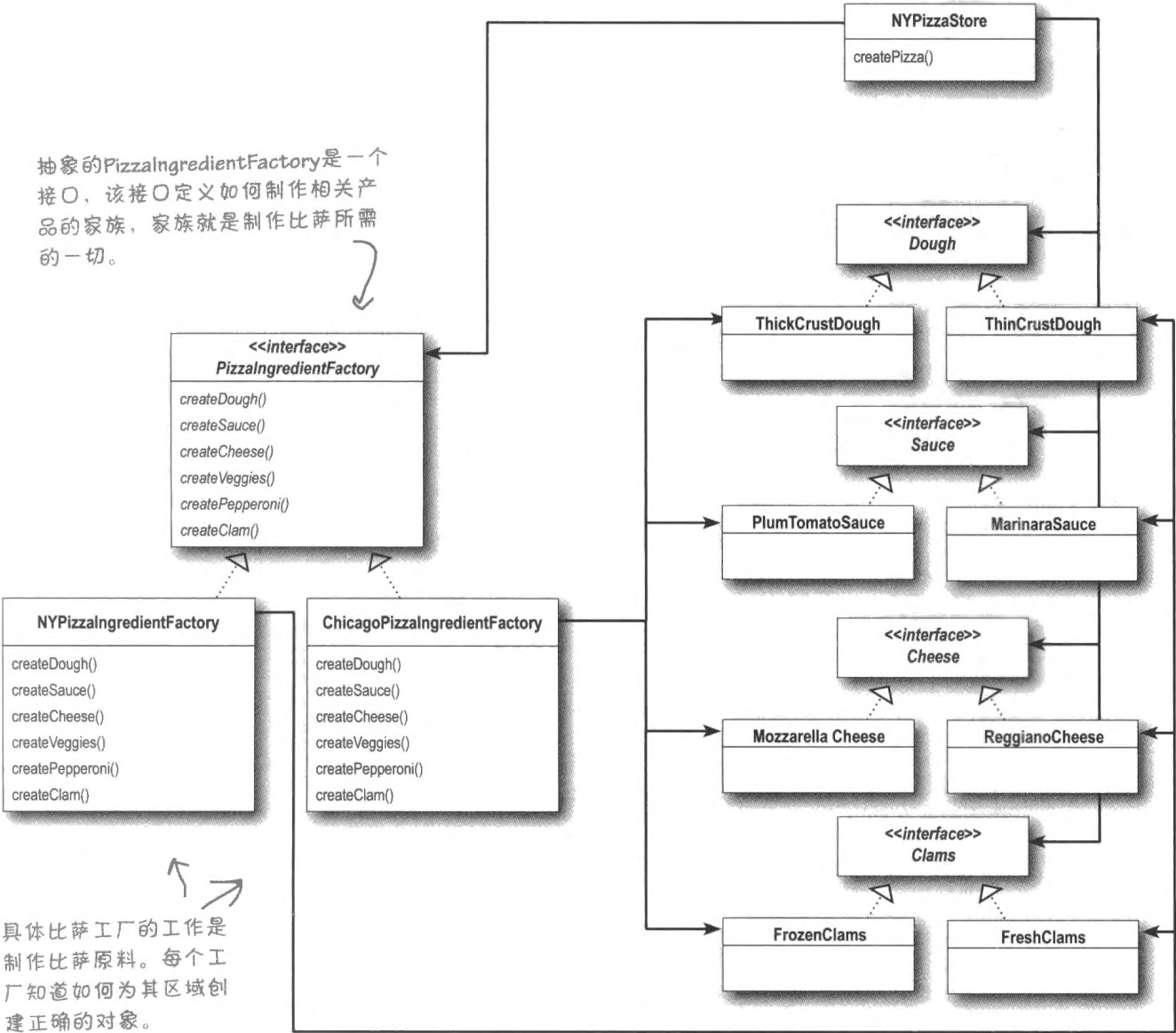
无疑，我们已经看到，抽象工厂允许客户使用一个抽象接口来创建一组相关的产品，而不需要懂得（或关心）实际生产的具体产品是什么。通过这样的方法，客户就从所有的特定具体产品解耦。我们来看看类图，了解其中的关系。



这是一张相当复杂的类图；我们从
PizzaStore的观点看看它：

抽象工厂的客户是比
萨店NYPizzaStore和
ChicagoStylePizzaStore
的两个实例。

抽象的PizzalngredientFactory是一个
接口，该接口定义如何制作相关产
品的家族，家族就是制作比萨所需
的一切。



具体比萨工厂的工作是
制作比萨原料。每个工
厂知道如何为其区域创
建正确的对象。

对于这个产品家族，每个工厂
有不同的实现。



我注意到，抽象工厂中的每个方法实际上看起来像工厂方法（`createDough()`、`createSauce()`等）。每个方法都被声明为抽象的，子类覆盖它，以创建一些对象。这不是工厂方法吗？

工厂方法是不是潜伏在抽象工厂里面？

没错！是的，抽象工厂的方法经常实现为工厂方法。这可以理解，对吧？抽象工厂的工作是定义一个接口，这个接口创建一组产品。这个接口的每个方法负责创建一个具体产品，我们实现抽象工厂的子类，以提供这些实现。因此，在抽象工厂中，用工厂方法来实现生产方法，是相当自然的方式。



模式曝光

本周的访谈：

工厂方法和抽象工厂，彼此之间

HeadFirst: 哇！一次访谈两个模式。这可是我们头一遭！

工厂方法: 呵呵，你知道，我好像不太喜欢别人把我和抽象工厂混为一谈。不能因为我们都是工厂模式，所以就不应该有自己专属的访谈。

HeadFirst: 别生气，我们要一起访谈你们的目的是，帮助读者搞清楚你们之间谁是谁。你们的确有相似之处，听说人们有时会把你们搞混。

抽象工厂: 这是真的，有些时候我被误认为是工厂方法。工厂方法，我知道你也有类似困扰。我们都很擅长从特定实现解耦应用，只是做法不同。因此，我能够理解为什么人们有时把我们搞混。

工厂方法: 好吧，这还是让我很不爽。毕竟，我用类来创建，而你用对象。这是完全不同的！

HeadFirst: 工厂方法，关于这个，能够请你多解释一些吗？

工厂方法: 当然可以。抽象工厂与我都创建对象，这是我们的工作。但是我是通过继承……

抽象工厂: ……而我是通过对象组合。

工厂方法: 对。这意味着，用工厂方法创建对象，你需要扩展一个类，并为工厂方法提供实现。

HeadFirst: 这个工厂方法做什么？

工厂方法: 当然是创建对象了！我的意思是，工厂方法模式的整体思想就是使用子类来创建。这样，客户只需要知道所用的抽象类型，子类操心具体类型。因此，换句话说，我保持客户从具体类型解耦。

抽象工厂: 我也做得到，只是我的做法不同。

HeadFirst: 继续，抽象工厂……你说了一些关于对象组合的事？

抽象工厂: 我为创建产品家族提供了一个抽象。这个类型的子类定义如何生产这些产品。要使用这个工厂，先实例化一个工厂，并给它传入一些针对抽象类型编写的代码。因此，像工厂方法一样，我们的客户从所用的实际具体产品解耦。

HeadFirst: 噢，我了解了，因此你的另一个优点是把一组相关的产品集结起来。

抽象工厂: 没错。

HeadFirst: 那要是你需要扩展这组相关产品，比如说，添加另一个产品，怎么办？难道不需要改变接口吗？

抽象工厂: 确实有这个问题，如果添加新产品，我的接口不得不改变，我知道大家不喜欢这么做……

工厂方法: <窃笑>

抽象工厂: 工厂方法，你偷笑什么？

工厂方法: 拜托，这是很严重的！改变接口，意味着你不得不深入内部并改变每个子类的接口！这听起来像是很繁重的工作。

抽象工厂: 是的，但是我需要一个大接口，因为我被用来创建整个产品家族。你只是创建一个产品，因此你根本不需要一个大接口，你只需要一个方法。

HeadFirst: 抽象工厂，我听说你经常使用工厂方法来实现你的具体工厂？

抽象工厂: 是的，我承认，我的具体工厂经常实现工厂方法来创建它们的产品。不过对我来说，这些具体工厂纯粹是用于创建产品而已……

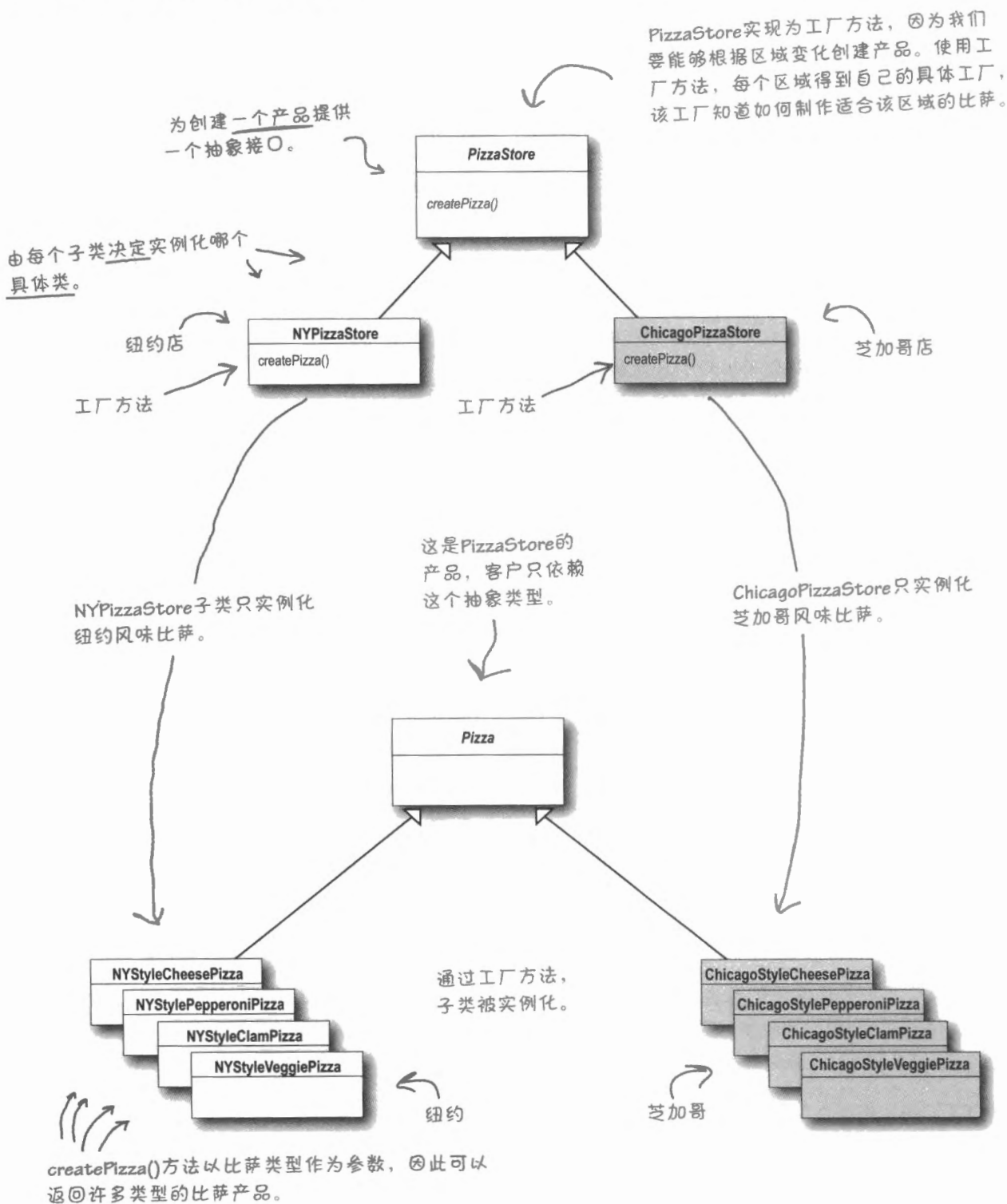
工厂方法: ……对我来说，我经常在抽象创建者中实现代码，会使用子类创建的具体类型。

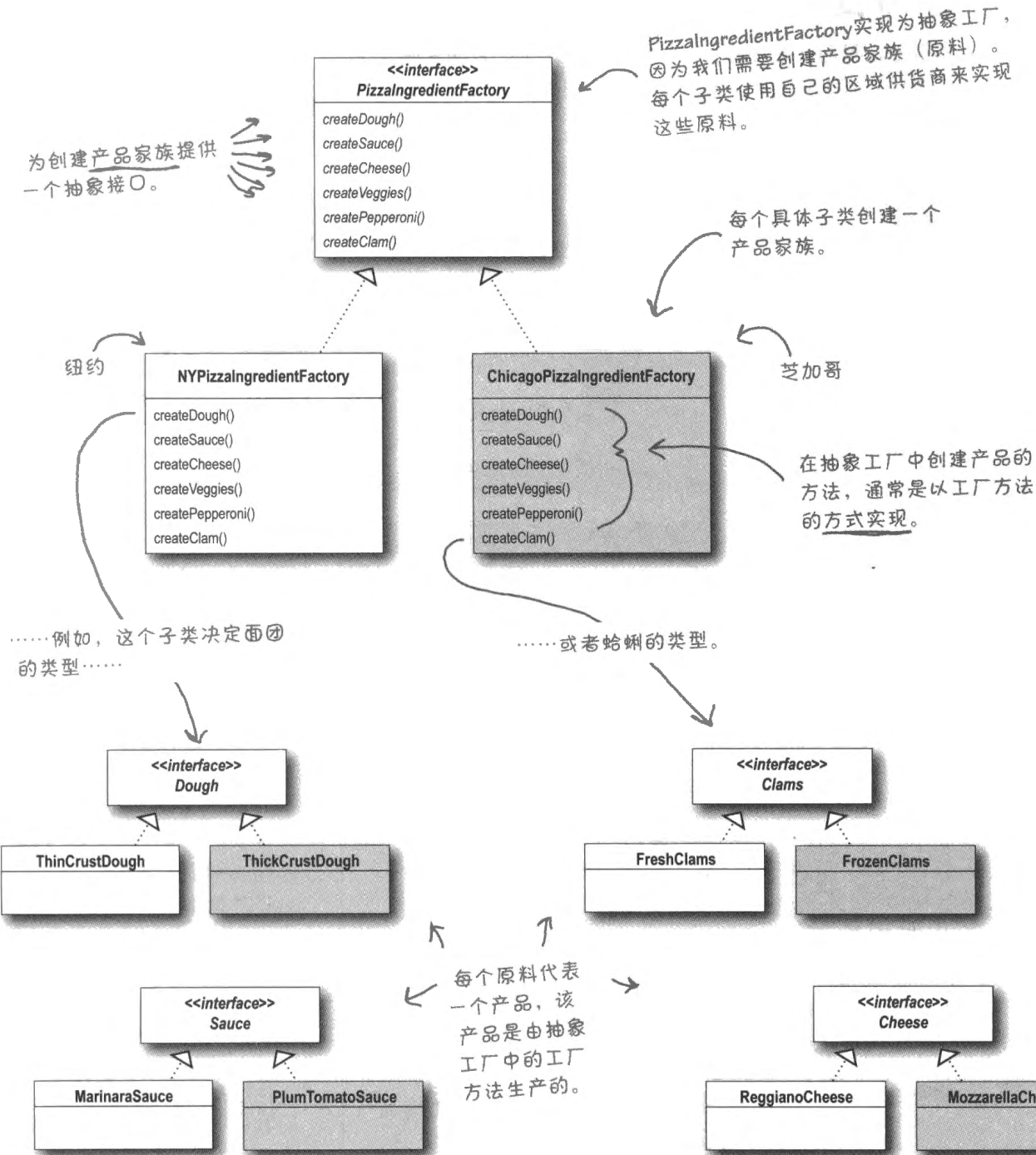
HeadFirst: 听起来你们都擅长所做的事情。我确信人们喜欢有选择的余地，毕竟，工厂这么有用，大家希望在各种不同的情况下都能够使用工厂。无论是使用工厂方法还是抽象工厂，你们两个都封装了对象的创建，以保持应用松耦合，降低对实现的依赖，这真的很棒。节目结束前，请两位各说几句话好吗？

抽象工厂: 谢谢。大家要记得我，抽象工厂，任何时候你需要创建产品家族，以及想确保你的客户创建相互关联的产品时，用我。

工厂方法: 我是工厂方法。用我将你的客户代码从你需要实例化的具体类解耦，或者如果你事先不知道会需要哪些具体类时，用我只需要子类化并实现工厂方法！

比较工厂方法和抽象工厂



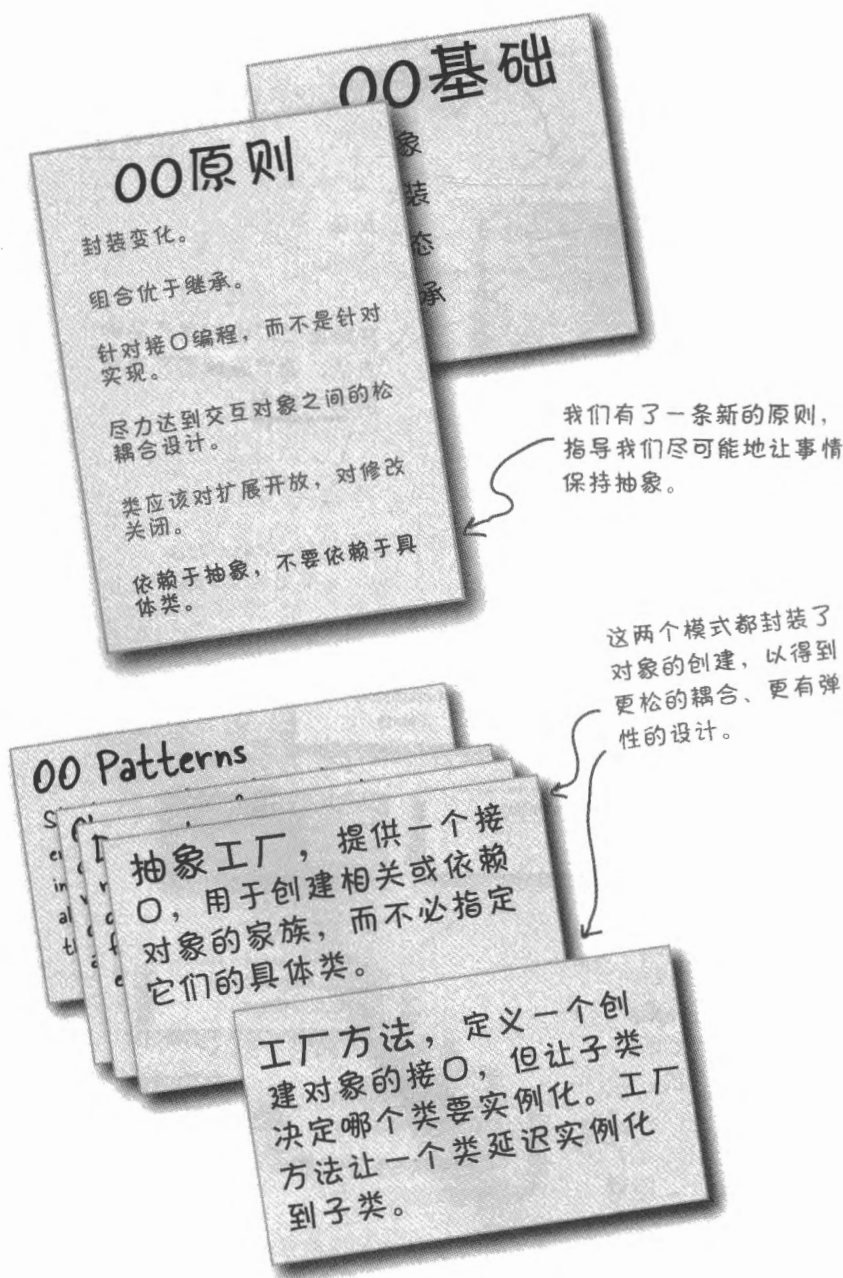


产品子类创建一组平行的产品家族。这里有纽约原料家族和芝加哥家族。



你的设计工具箱中的工具

本章中，我们又添加两个工具到工具箱：工厂方法和抽象工厂。两个模式都封装对象的创建，并允许代码从具体类型解耦。



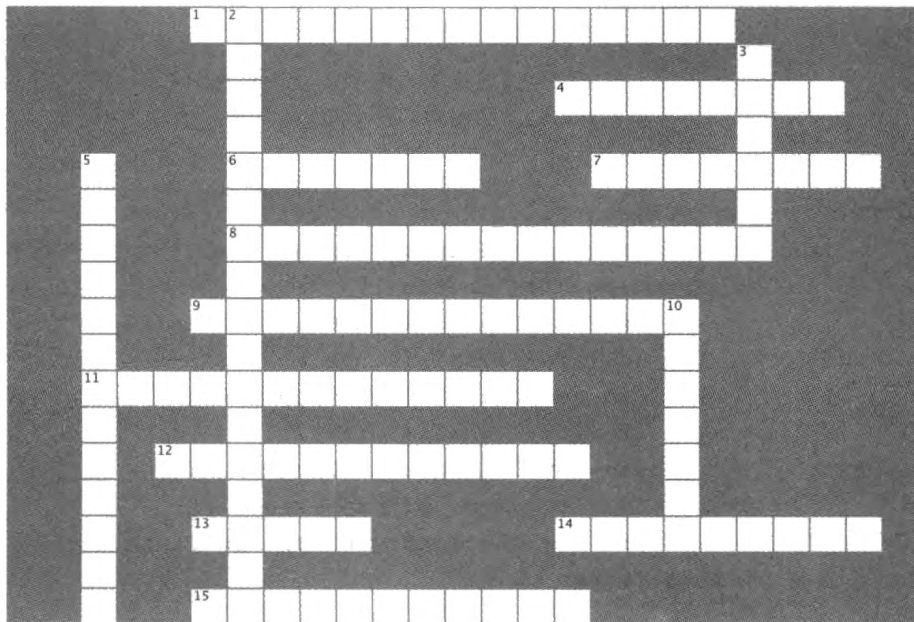
要点

- 所有工厂都封装对象的创建。
- 简单工厂虽然不是真正的设计模式，但依然可以作为一个简单的方法，将客户从具体类解耦。
- 工厂方法靠继承：对象创建被委托给子类，子类实现工厂方法来创建对象。
- 抽象工厂靠对象组合：对象创建在工厂接口暴露的方法中实现。
- 所有工厂模式都通过减少应用对具体类的依赖，促进了松耦合。
- 工厂方法的意图，是允许一个类延迟实例化到其子类。
- 抽象工厂的意图，是创建相关对象家族，不必依赖于其具体类。
- 依赖倒置原则指导我们避免依赖具体类型，尽量依赖抽象。
- 工厂是强有力的技巧，让我们针对抽象编码，而不是针对具体类。



设计模式填字游戏

这一章很长。抓起一片比萨，做下面这个填字游戏放松一下。答案词汇都来自本章。



横排

1. 在工厂方法中，每个加盟店是一个_____。
4. 在工厂方法中，谁决定要实例化哪个类？
6. 工厂方法模式中PizzaStore的模式。
7. 所有纽约风味比萨使用这种奶酪。
8. 在抽象工厂中，每个原料工厂是一个_____。
9. 当你使用 new，你是在针对_____编程。
11. createPizza()是一个_____。
12. Joel喜欢这种比萨。
13. 在工厂方法中， PizzaStore和具体比萨都依赖于这个抽象。
14. 当一个类从具体类实例化对象时，它_____于该对象。
15. 所有工厂模式允许我们_____对象创建。

竖排

2. 在简单工厂和抽象工厂中我们用_____，在工厂方法中我们用继承。
3. 抽象工厂 创建一个产品的_____。
5. 不是一个真正的工厂模式，尽管比较方便。
10. Ethan喜欢这种比萨。



削尖你的铅笔 答案

我们已经解决了NYPizzaStore，还剩下两个准备加盟的要解决。编写芝加哥和加州的PizzaStore实现：

这两个店都和纽约店几乎一样……
只是创建不同种类的比萨。

```
public class ChicagoPizzaStore extends PizzaStore {
    protected Pizza createPizza(String item) {
        if (item.equals("cheese")) {
            return new ChicagoStyleCheesePizza();
        } else if (item.equals("veggie")) {
            return new ChicagoStyleVeggiePizza();
        } else if (item.equals("clam")) {
            return new ChicagoStyleClamPizza();
        } else if (item.equals("pepperoni")) {
            return new ChicagoStylePepperoniPizza();
        } else return null;
    }
}
```

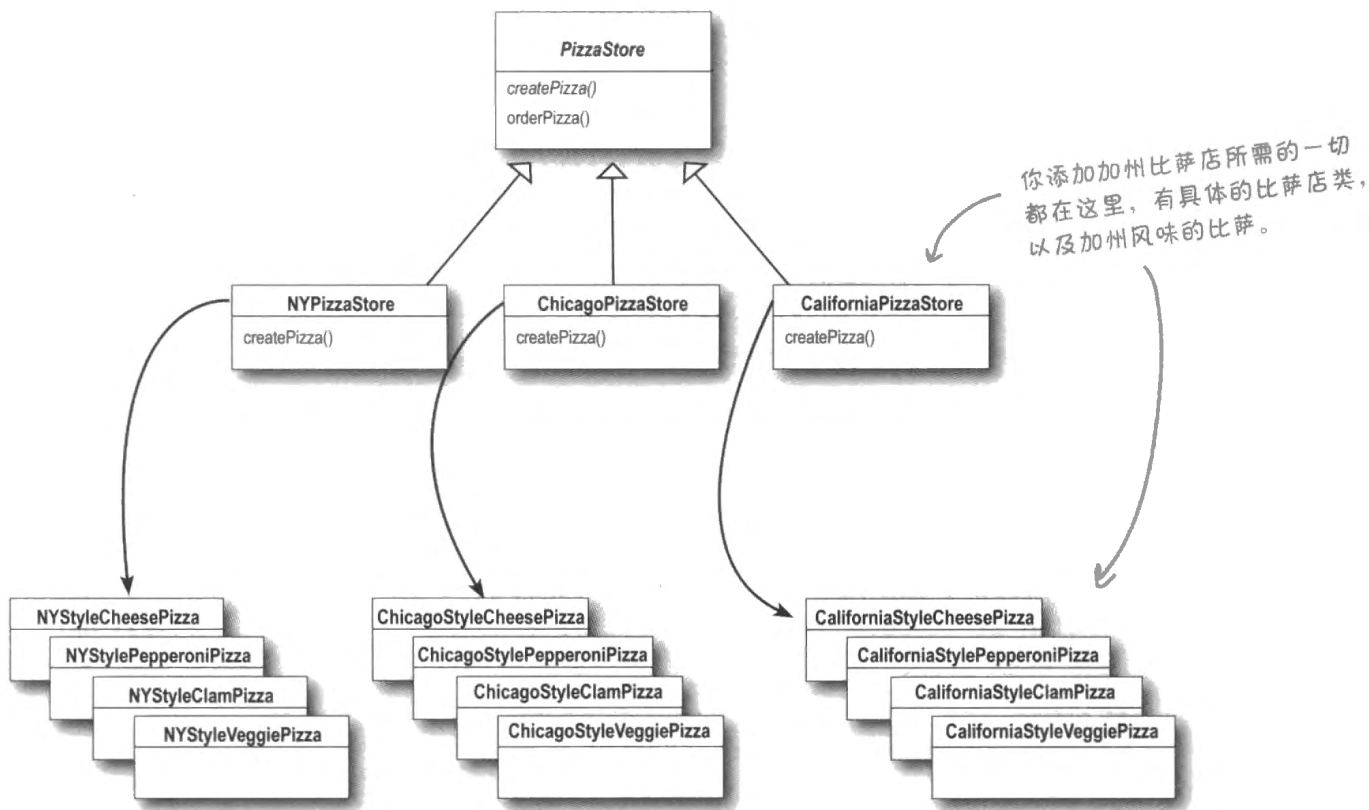
对于芝加哥比萨店，
我们只要确认创建芝
加哥风味的比萨……

```
public class CaliforniaPizzaStore extends PizzaStore {
    protected Pizza createPizza(String item) {
        if (item.equals("cheese")) {
            return new CaliforniaStyleCheesePizza();
        } else if (item.equals("veggie")) {
            return new CaliforniaStyleVeggiePizza();
        } else if (item.equals("clam")) {
            return new CaliforniaStyleClamPizza();
        } else if (item.equals("pepperoni")) {
            return new CaliforniaStylePepperoniPizza();
        } else return null;
    }
}
```

……而对于加州
比萨店，我们创
建加州风味的
比萨。

设计谜题解答

我们需要另一种比萨，以符合疯狂的加州人的口味（当然，这里的疯狂是指好的那一方面）。请绘制出另一组平行的类，把加州区域纳入PizzaStore中。



OK，发挥你的想象力，找出五个“最奇特”的东西加入比萨中，写下来。
然后你就可以准备到加州去开比萨店了！

这是我们的
建议……

带烤蒜的土豆泥

BBQ酱

朝鲜蓟心

M&M巧克力

花生



削尖你的铅笔

答案

我们假装你从未听说过OO工厂。下面是一个“依赖性很强”、没有使用工厂的PizzaStore版本。我们需要你来数一数，这个类所依赖的具体比萨类的数目。如果添加加州风味比萨到这个PizzaStore，又会依赖多少个类？答案如下。

```
public class DependentPizzaStore {

    public Pizza createPizza(String style, String type) {
        Pizza pizza = null;
        if (style.equals("NY")) {
            if (type.equals("cheese")) {
                pizza = new NYStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new NYStyleVeggiePizza();
            } else if (type.equals("clam")) {
                pizza = new NYStyleClamPizza();
            } else if (type.equals("pepperoni")) {
                pizza = new NYStylePepperoniPizza();
            }
        } else if (style.equals("Chicago")) {
            if (type.equals("cheese")) {
                pizza = new ChicagoStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new ChicagoStyleVeggiePizza();
            } else if (type.equals("clam")) {
                pizza = new ChicagoStyleClamPizza();
            } else if (type.equals("pepperoni")) {
                pizza = new ChicagoStylePepperoniPizza();
            }
        } else {
            System.out.println("Error: invalid type of pizza");
            return null;
        }
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }
}
```

处理所有纽约
风味比萨。

处理所有芝加哥
风味比萨。

你可以把答案写
在这里：

8 数目

12 加上加州风味后的
数目



削尖你的铅笔 答案

编写 ChicagoPizzaIngredientFactory。你可以在你的实现中引用下面的类：

```
public class ChicagoPizzaIngredientFactory
    implements PizzaIngredientFactory
{
    public Dough createDough() {
        return new ThickCrustDough();
    }

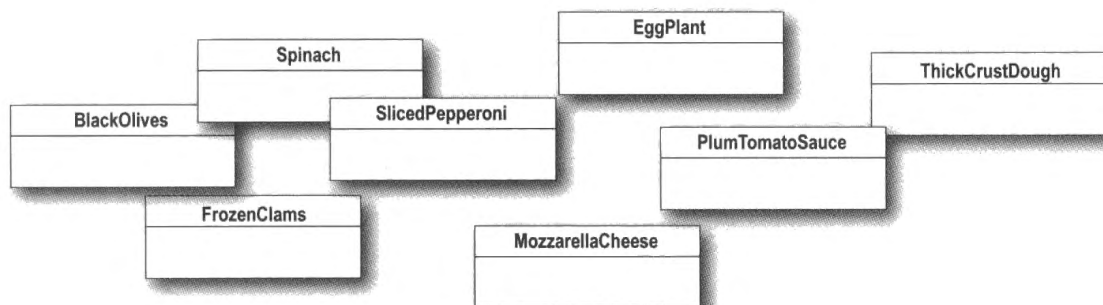
    public Sauce createSauce() {
        return new PlumTomatoSauce();
    }

    public Cheese createCheese() {
        return new MozzarellaCheese();
    }

    public Veggies[] createVeggies() {
        Veggies veggies[] = { new BlackOlives(),
                               new Spinach(),
                               new Eggplant() };
        return veggies;
    }

    public Pepperoni createPepperoni() {
        return new SlicedPepperoni();
    }

    public Clams createClam() {
        return new FrozenClams();
    }
}
```





设计模式填字游戏解答

这一章很长。抓起一片比萨，做下面这个填字游戏放松一下。答案词汇都来自本章。
答案如下。

